Lecture 04
Model-based Recommender Systems (part 2)

Euhyun Moon, Ph.D.
Machine Learning Systems (MLSys) Lab
Computer Science and Engineering
Sogang University



Slides adapted from D. Jurafsky: Neural Networks, https://web.stanford.edu/class/cs124, Stanford University



Learning Embeddings – part 1

What is Machine Learning?

- Machine learning is about *Optimization*

- Three key components:
  1. Training Data $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
  2. Loss function $\mathcal{L}$
  3. Model $f_\theta(x)$

- Optimize $f_\theta(x)$ on D w.r.t loss function $\mathcal{L}$:
  - Find the parameter θ that minimizes the expected loss on the training data

$$\min_f J(f) = \min_f \frac{1}{\mathrm{n}} \sum_{i=1}^{\mathrm{n}} \mathcal{L}(f_\theta(x_i), y_i)$$

Two Dominant ML Paradigms

- **Supervised learning**:
  - Given "labeled data" $\{x, y\}$, learn $f(x) = y$
  - Ex: classification, regression
    - In linear regression, the model $f_\theta(x) = Wx + b$
    - Parameters are $\theta = \{W, b\}$
    - The loss function is mean square error (MSE)

- **Unsupervised learning**:
  - Given only "unlabeled data" $\{x\}$, learn $f(x)$
  - Ex: dimensionality reduction, clustering
    - In SVD, the model is $f(x) = \hat{x} = VVTx$ where V is right singular vectors of input matrix
    - The loss function is L2 loss: $L(x, \hat{x}) = \sum ||x - \hat{x}||^2$

Input Feature Vectors

- All ML methods work with the input feature vectors $\{x_1, x_2, \ldots, x_n\}$ and almost all of them require input features to be numerical

- From ML perspective, there are four types of features:
  - Numerical (continues or discrete)
    - Continues: height
    - Discrete: the number of people in a class
  - Categorical (ordinal or nominal)
    - Ordinal: level = {beginner, intermediate, advanced}
    - Nominal: gender = {male, female}, color = {red, blue, green}
  - Time series
    - Average of home sale price over years
  - Text
    - Bag of words

Categorical Features

- There are two ways to encode categorical variable:
  - Integer encoding
  - One-hot encoding (and multi-hot encoding)
- Consider the following movie dataset:

Title	provider	IMDB genres	Release year	IMDB rating
Stranger Things	Netflix	drama, fantasy, horror	2016	8.7
Cocomelon	Prime Video	animation, comedy, family	2019	4.7
100 Foot Waves	HBO Max	documentary, sport	2021	8.1
I, Tonya	Hulu	biography, drama, comedy	2017	7.5

Integer Encoding

- Assigns each category value with an integer
  - *provider := [Netflix, Prime Video, HBO Max, Hulu]*, we assign them integers 1, 2, 3 and 4, respectively

Title	provider	IMDB genres	Release year	IMDB rating
Stranger Things	Netflix	drama, fantasy, horror	2016	8.7
Cocomelon	Prime Video	animation, comedy, family	2019	4.7
100 Foot Waves	HBO Max	documentary, sport	2021	8.1
I, Tonya	Hulu	biography, drama, comedy	2017	7.5

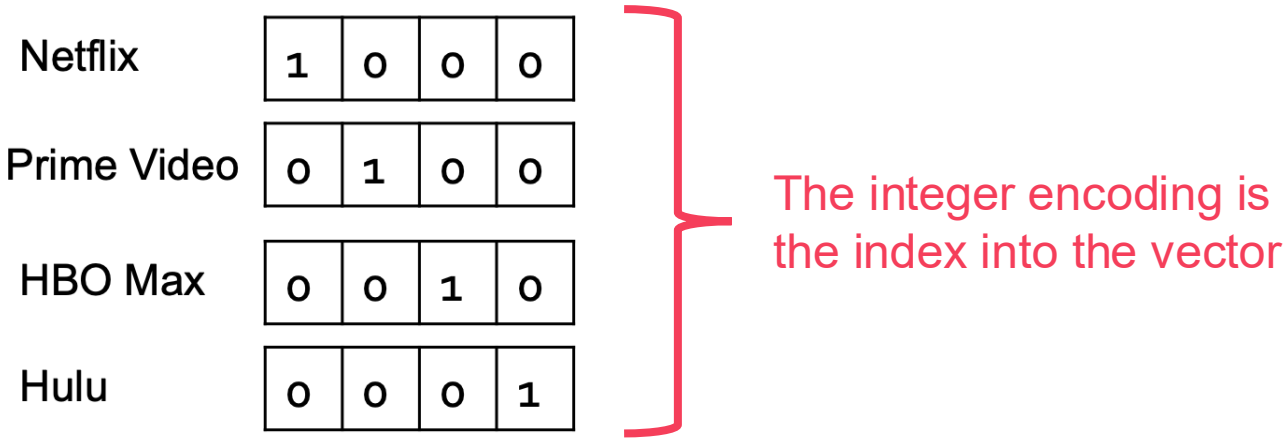


Title	provider	IMDB genres	Release year	IMDB rating
Stranger Things	1	drama, fantasy, horror	2016	8.7
Cocomelon	2	animation, comedy, family	2019	4.7
100 Foot Waves	3	documentary, sport	2021	8.1
I, Tonya	4	biography, drama, comedy	2017	7.5

- Pros: Dense representation
- Cons: It implies ordering between different categories
  - *Netflix < Prime Video < HBO Max < Hulu*
- Makes more sense to use it for ordinal variables
  - Such as “Education”= {Diploma, Undergrad, Masters, Phd}
  - But still it implies values are equally spaced out

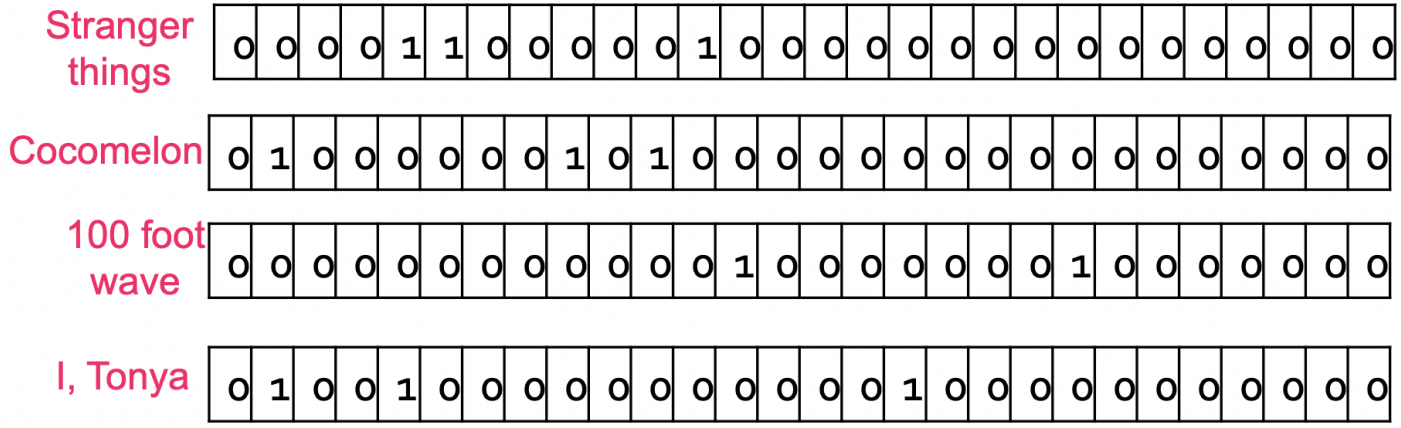
One-hot Encoding

- First, do integer encoding, then create a binary vector that represents the numerical values
  - Ex: following integer encoding on provider:
    - Netflix → 1, Prime Video → 2, HBO Max → 3, Hulu → 4
  - create a binary vector of length 4 for each value:



Multi-hot Encoding

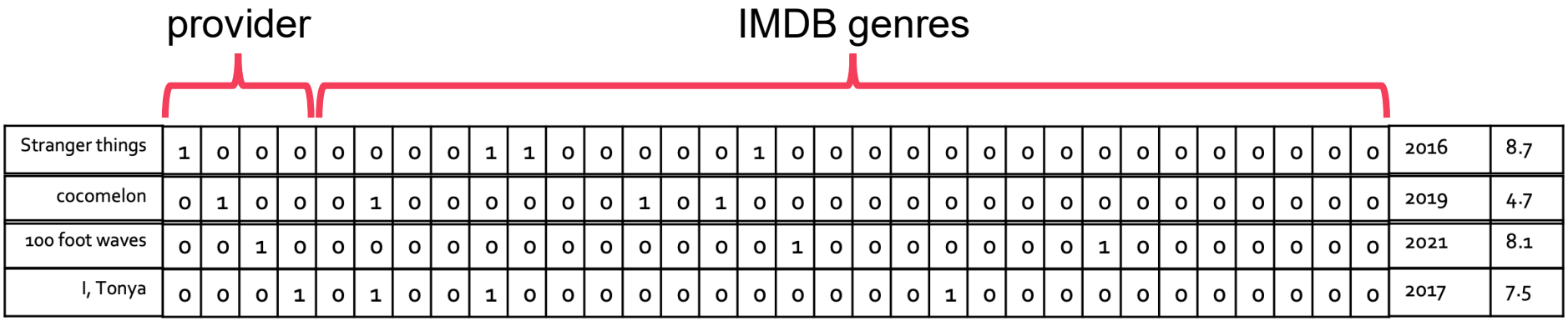
- An extension of one-hot encoding when categorical variable can take multiple values at the same time
  - Ex: There are 28 distinct IMDB genres
  - A movie can take multiple genres
    - Ex: *stranger things is drama, fantasy, horror*



IMDB genres

```
[(1, 'Action'),
 (2, 'Comedy'),
 (3, 'Short'),
 (4, 'Western'),
 (5, 'Drama'),
 (6, 'Horror'),
 (7, 'Music'),
 (8, 'Thriller'),
 (9, 'Animation'),
 (10, 'Adventure'),
 (11, 'Family'),
 (12, 'Fantasy'),
 (13, 'Sport'),
 (14, 'Romance'),
 (15, 'Crime'),
 (16, 'Sci-Fi'),
 (17, 'Biography'),
 (18, 'Musical'),
 (19, 'Mystery'),
 (20, 'History'),
 (21, 'Documentary'),
 (22, 'Film-Noir'),
 (23, 'News'),
 (24, 'Game-Show'),
 (25, 'Reality-TV'),
 (26, 'War'),
 (27, 'Talk-Show'),
 (28, 'Adult')]
```

Applying Encodings on Movies Dataset



Stranger things	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2016	8.7
cocomelon	0	1	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2019	4.7
100 foot waves	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	2021	8.1
I, Tonya	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	2017	7.5

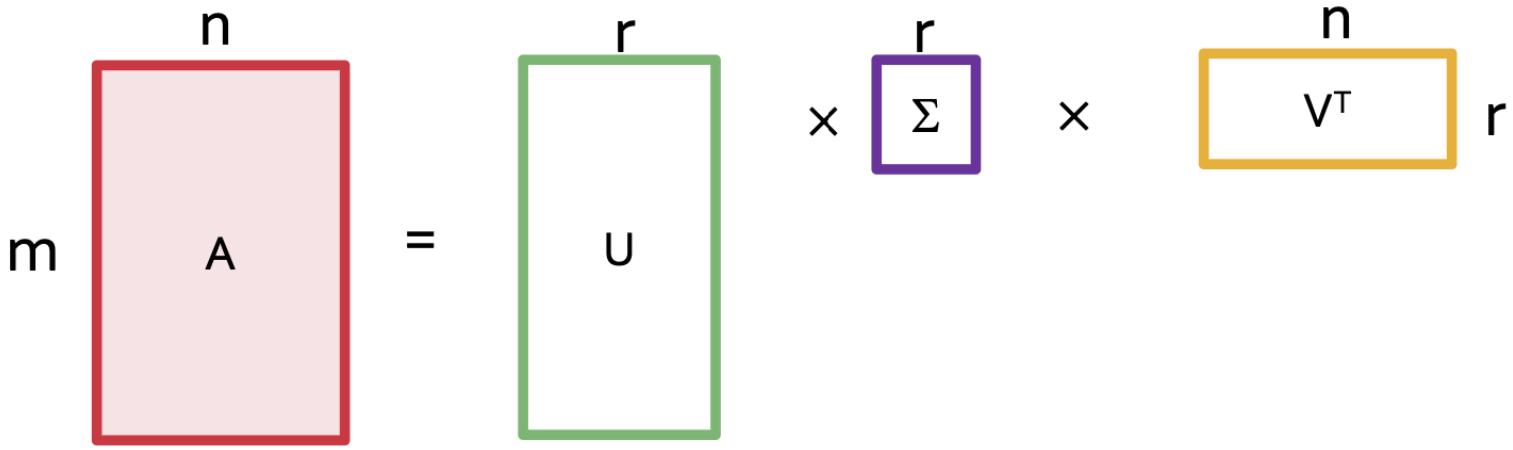
- Data dimensions increased from 5 to 35. It will blow up to thousands or a million if we multi-hot encode title!
- One-hot and multi-hot encodings are not practical for features with large value sets

From Encoding to Embedding

- One-hot/multi-hot encodings:
  - pros: simple, robust, the observation that simple models trained on huge amounts of data outperform complex systems trained on less data
  - cons: sparse and high dimensional, don't capture semantic similarity
- In a corpus of documents with one million distinct words:
  - high dimensional: multi-hot encodings are 1-million dimensional
  - Sparse: an average document contains 500 words therefore the multi- hot encodings are > 99.95% sparse
  - lack of semantic: encoding of two words 'good' and 'great' are as different as encoding of 'good' and 'bad'!
- An embedding is a translation of a high-dimensional vector into a low-dimensional space. An embedding is a:
  - Dense representation (floating-point value)
  - Low-dimensional vector
  - Captures semantic similarity

Review: SVD as an Embedder

- Standard dimensionality reduction methods
  - Singular value decompositions (SVD)



- **A**: **Input data matrix**: m × n matrix (e.g., m documents, n terms)
  - r : rank of the matrix **A** – often r < min(m,n)
- **U**: **Left singular vectors**: m × r matrix (m documents, r concepts)
- **Σ**: **Singular values**: r × r diagonal matrix (strength of each 'concept')
- **V**: **Right singular vectors**: n × r matrix (n terms, r concepts)

Review: SVD as an Embedder

- ***U, V***: column orthonormal
  - $\boldsymbol{U^T U = I}$; $\boldsymbol{V^T V = I}$ (***I***: identity matrix)
  - Columns are orthogonal unit vectors hence they define an r-dimensional subspace
    - ***U*** defines an r-dimensional subspace in $\boldsymbol{R^m}$
    - ***V*** defines an r-dimensional subspace in $\boldsymbol{R^n}$

- Projecting ***A*** onto ***V*** and ***U*** produces embeddings:
  - **Since $A = U\Sigma V^T$ then $AV = U\Sigma$ are row embeddings**
  - **Since $A = U\Sigma V^T$ then $U^T A = \Sigma V^T$ are column embeddings**

Review: SVD as an Embedder

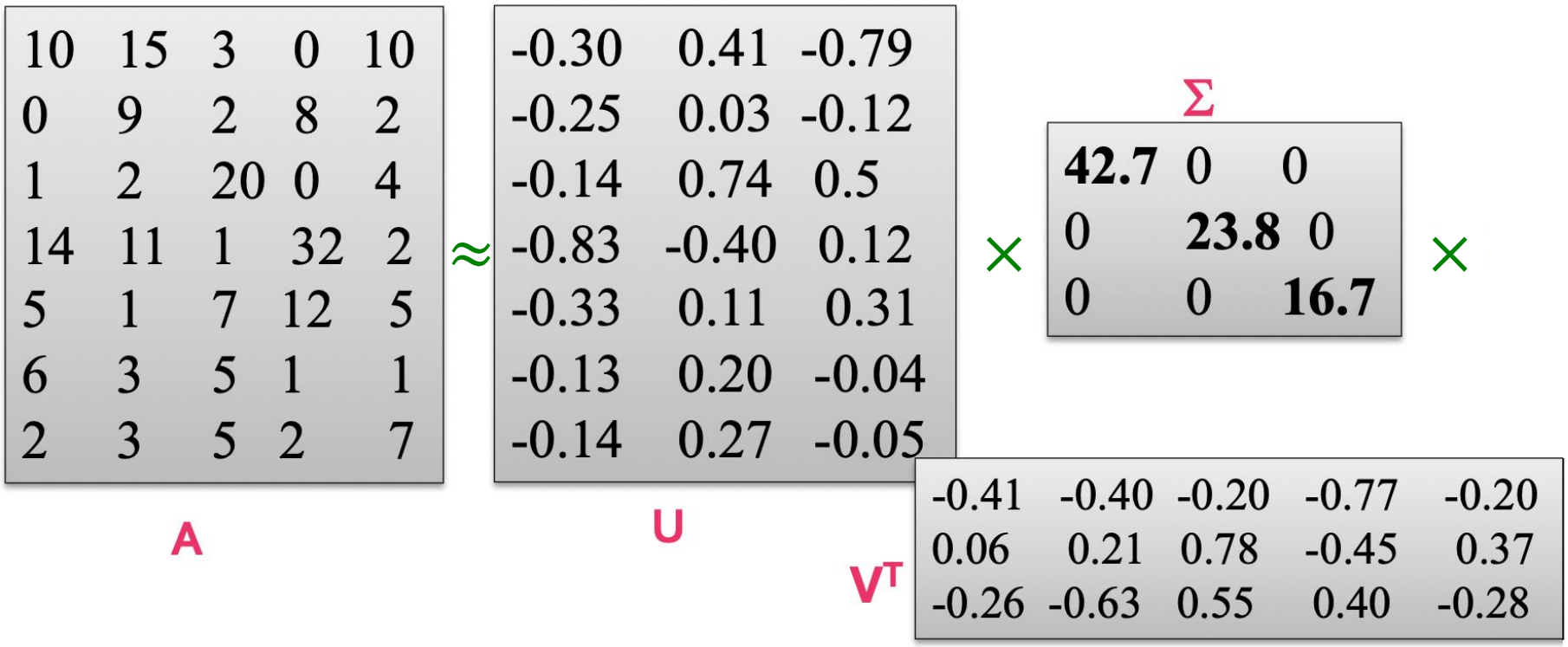
- Ex: compute document & word embeddings
- **Step 1**: given a corpus of documents, convert it to BOW vectors → get a term-document matrix
  - Use term frequencies(tf), or normalize using tf-idf

	data	science	spark	Stanford	learning
document 1	10	15	3	0	10
document 2	0	9	2	8	2
document 3	1	2	20	0	4
document 4	14	11	1	32	2
document 5	5	1	7	12	5
document 6	6	3	5	1	1
document 7	2	3	5	2	7

Review: SVD as an Embedder

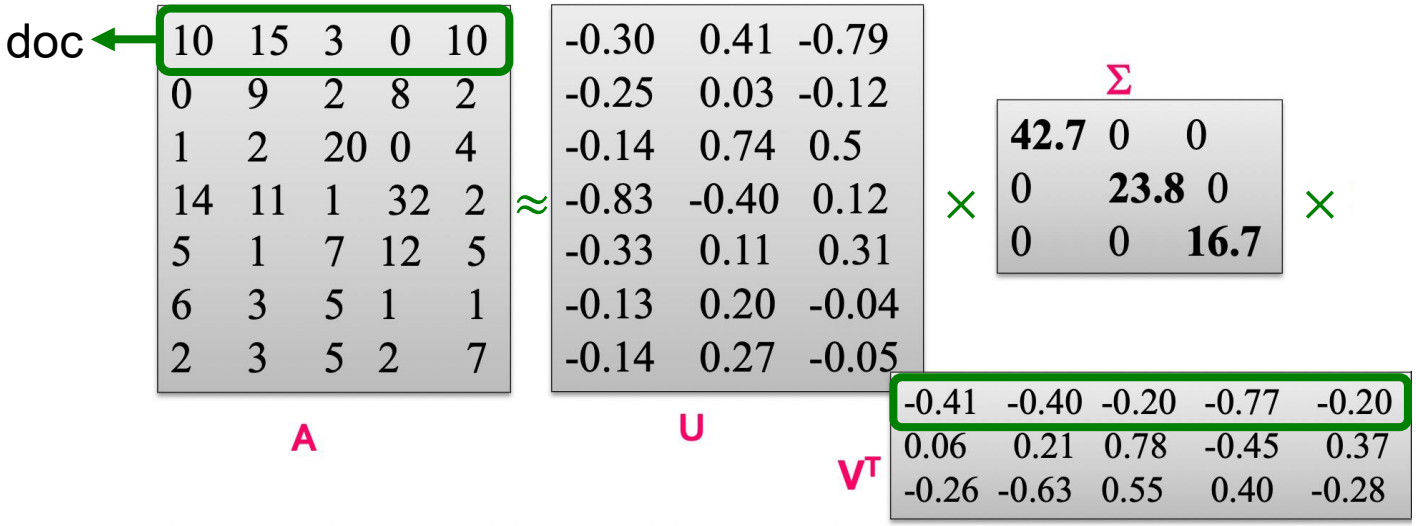
- **Step 2**: apply SVD on the term-document matrix and pick a value $r \leq rank(A)$

Here, we set r = 3



Review: SVD as an Embedder

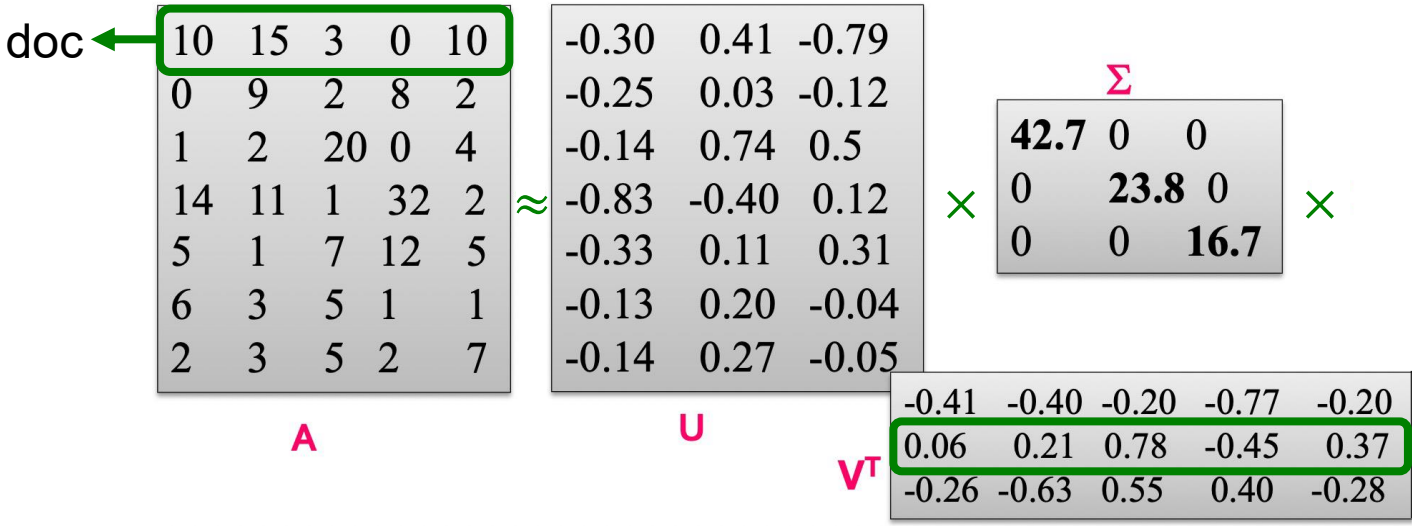
- **Step 3**: compute embedding of documents as embedding = [<doc, v1> , <doc, v2> , <doc, v3>]



- <doc, v1> = <[10,15,3,0,10] , v1>= -12.7

Review: SVD as an Embedder

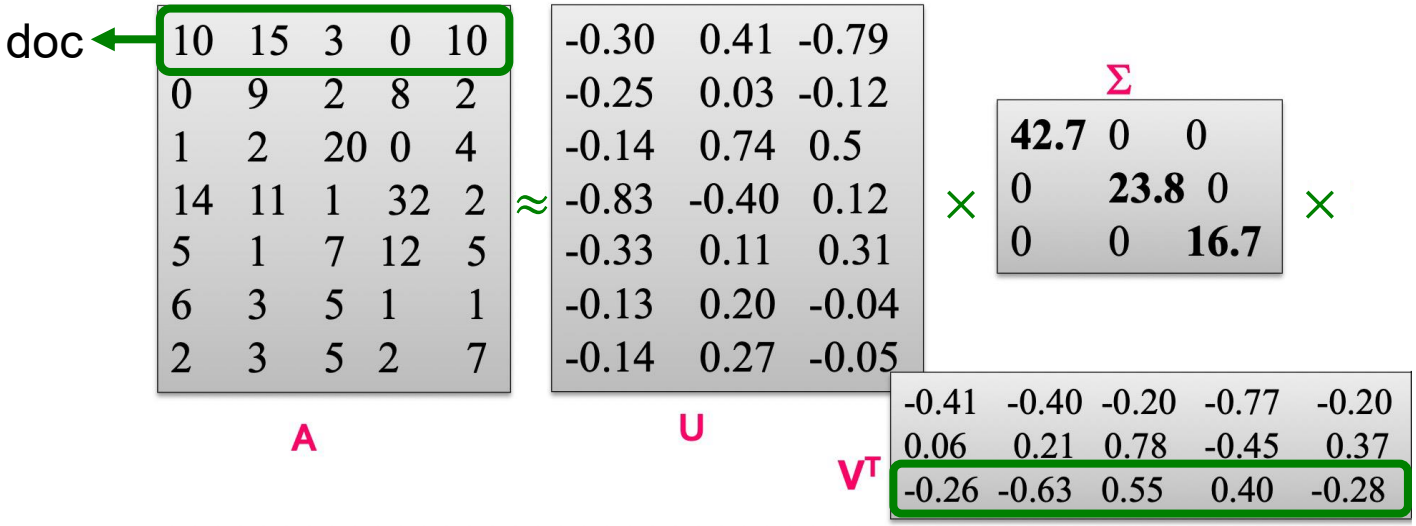
- **Step 3**: compute embedding of documents as embedding = [<doc, v1> , <doc, v2> , <doc, v3>]



- <doc, v1> = <[10,15,3,0,10] , v1>= -12.7
- <doc, v2> = <[10,15,3,0,10] , v2> = 9.79

Review: SVD as an Embedder

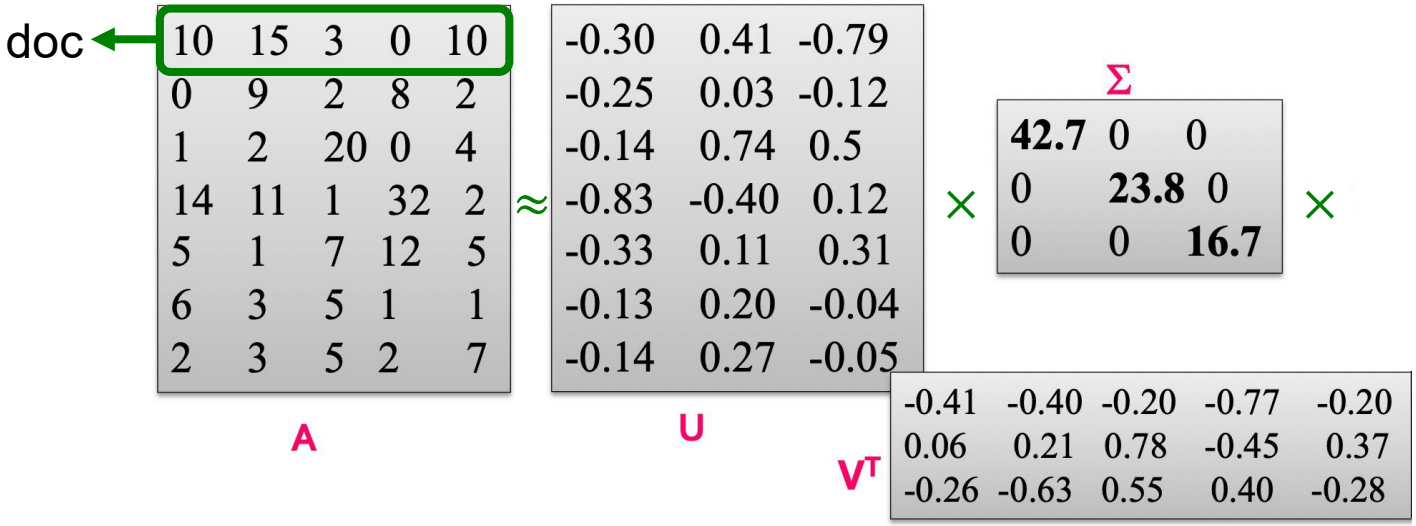
- **Step 3**: compute embedding of documents as embedding = [<doc, v1> , <doc, v2> , <doc, v3>]



- <doc, v1> = <[10,15,3,0,10] , v1>= -12.7
- <doc, v2> = <[10,15,3,0,10] , v2> = 9.79
- <doc, v3> = <[10,15,3,0,10] , v3>= -13.9

Review: SVD as an Embedder

- **Step 3**: compute embedding of documents as embedding = [<doc, v1> , <doc, v2> , <doc, v3>]



- <doc, v1> = <[10,15,3,0,10] , v1>= -12.7
- <doc, v2> = <[10,15,3,0,10] , v2> = 9.79
- <doc, v3> = <[10,15,3,0,10] , v3>= -13.9

Embedding for document 1 = [-12.7, 9.79, -13.9]

Review: SVD as an Embedder

- SVD is impractical on real-world datasets
  - There are 0.5 billion wiki pages, and 4 billion words
  - SVD is computationally prohibitive, as it requires to load all data in memory
  - SVD is a linear embedder
  - Not utilizing sparsity
  - Orthonormality constraint is an overkill

From SVD to Neural Networks

- State of the art embedders are among **Neural Networks**

- Can we use neural networks to create non-linear embedding?

Learning Embeddings using Neural Networks

- We will work with three examples:
  1. Word embeddings produced by Word2Vec model
     - Converts one-hot encoding to dense embedding
     - Task independent and unsupervised
  2. Example of task specific embedding
     - converts one-hot encoding to embedding in supervised mode
  3. Autoencoders
     - learn short representation from a large feature vector

Learning Embeddings using Neural Networks

- We will work with three examples:
  1. **Word embeddings produced by Word2Vec model**
     - **Converts one-hot encoding to dense embedding**
     - **Task independent and unsupervised**
  2. Example of task specific embedding
     - converts one-hot encoding to embedding in supervised mode
  3. Autoencoders
     - learn short representation from a large feature vector

Word Embedding

- There are many techniques to learn word embeddings: Word2Vec, GloVe, BERT, fastText
- **Today's lecture**: **Word2Vec**
- Word2Vec was developed at Google in 2013
  - Mikolov et al. "Distributed Representations of Words and Phrases and their Compositionality" NeurIPS 2013
- Word2Vec is a statistical method for efficiently learning word embedding. It is unsupervised, and task independent.

Word2Vec

- Word2Vec comes in two architectures:
  - Continuous Bag-of-Words (CBOW)
  - Skip-Gram
    - We will discuss skip-gram model today
  - The two methods are very similar, both use a shallow neural network (only 1 hidden layer) to learn word representations
  - The key idea of word2Vec is that words with similar context have similar meanings
    - It learns embedding based on the usage of words

Word2Vec: Target and Context

- **The key idea**: The more often a word appears in the context of certain other words, the closer they are in meaning
- This is how we define context:
  - Set a window size (e.g., window = 2)
  - For any given word (aka target word), 2 words to its left & 2 words to its right are the context words
  - Window size is a hyperparameter

Word2Vec: Target and Context

- **The key idea**: The more often a word appears in the context of certain other words, the closer they are in meaning
- This is how we define context:
  - Set a window size (e.g., window = 2)
  - For any given word (aka target word), 2 words to its left & 2 words to its right are the context words
  - Window size is a hyperparameter

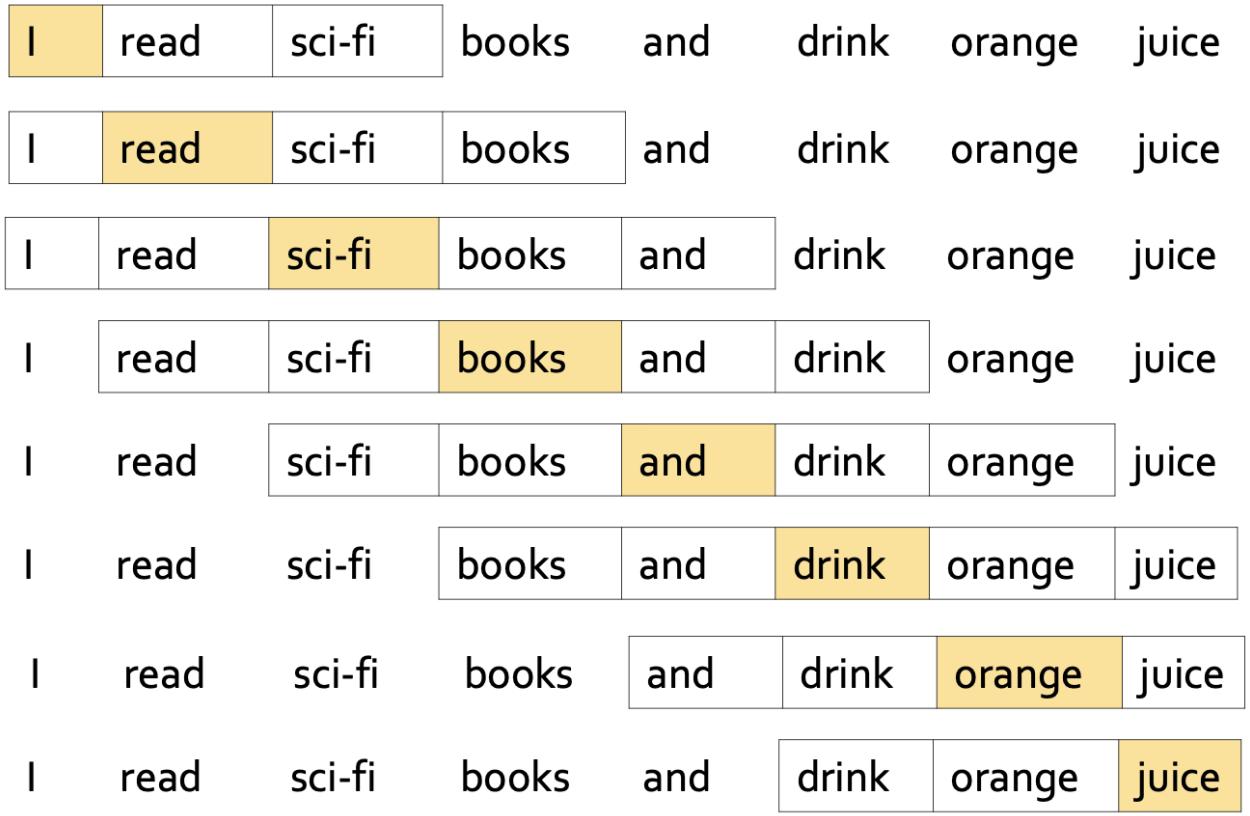
  - For example, in sentence "I read sci-fi books":
    - Target = "I" → context words = "read", "sci-fi" (no words to the left of "I")
    - Target = "read" → context words = "I", "sci-fi", "books"

Word2Vec: Target and Context

- **The key idea**: The more often a word appears in the context of certain other words, the closer they are in meaning
- This is how we define context:
  - Set a window size (e.g., window = 2)
  - For any given word (aka target word), 2 words to its left & 2 words to its right are the context words
  - Window size is a hyperparameter
  - For example, in sentence "I read sci-fi books":
    - Target = "I" → context words = "read", "sci-fi" (no words to the left of "I")
    - Target = "read" → context words = "I", "sci-fi", "books"
  - Given a document, we can slide the window from left to right and find all pairs of (target, context) words

Word2Vec: Target and Context

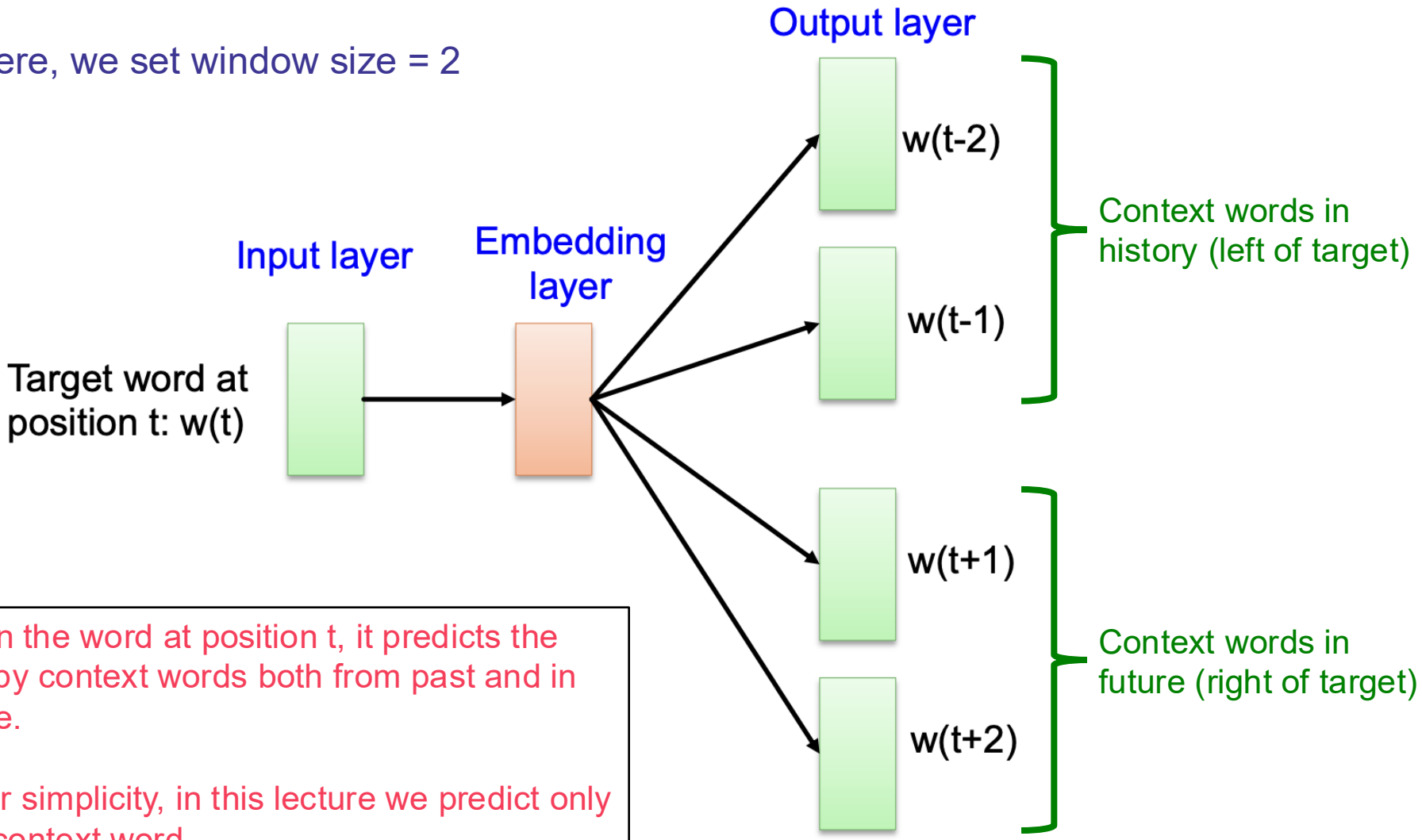
- Window size is a hyper-parameter
  - Here window size = 2
- The highlighted word is the target word
  - Other words in the box are context words



Word2Vec: Architecture

- Word2Vec is a 2 layers NN (i.e., only 1 hidden layer)
- Given one-hot encoding of the target word, it predicts context words
  - In our example, if target word = “I” then context words = “read”, “sci-fi”
  - Given one-hot encoding of word “I”, it predicts the context words
- If window size = N, the model predicts the N-grams words except the current word as it is the input to the model, hence the name skip-gram

Word2Vec: High Level Architecture

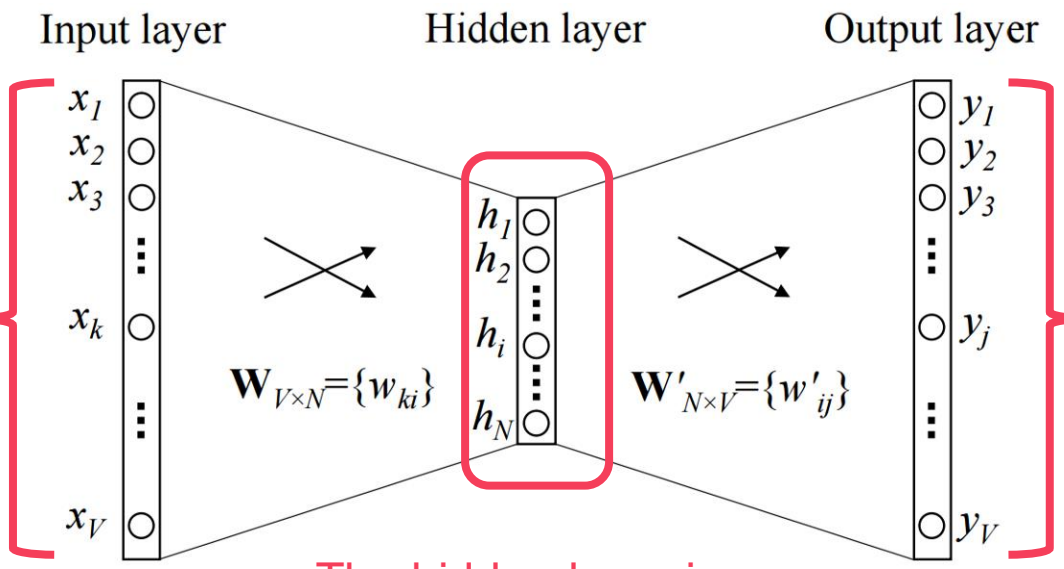


Given the word at position t, it predicts the nearby context words both from past and in future.

** For simplicity, in this lecture we predict only one context word.

Word2Vec: Architecture

- Word2Vec is a 2 layers NN (i.e., only 1 hidden layer)
- Let V = size of vocabulary and N = embedding dimension
  - We build vocabulary by extracting 1-gram tokens from corpus



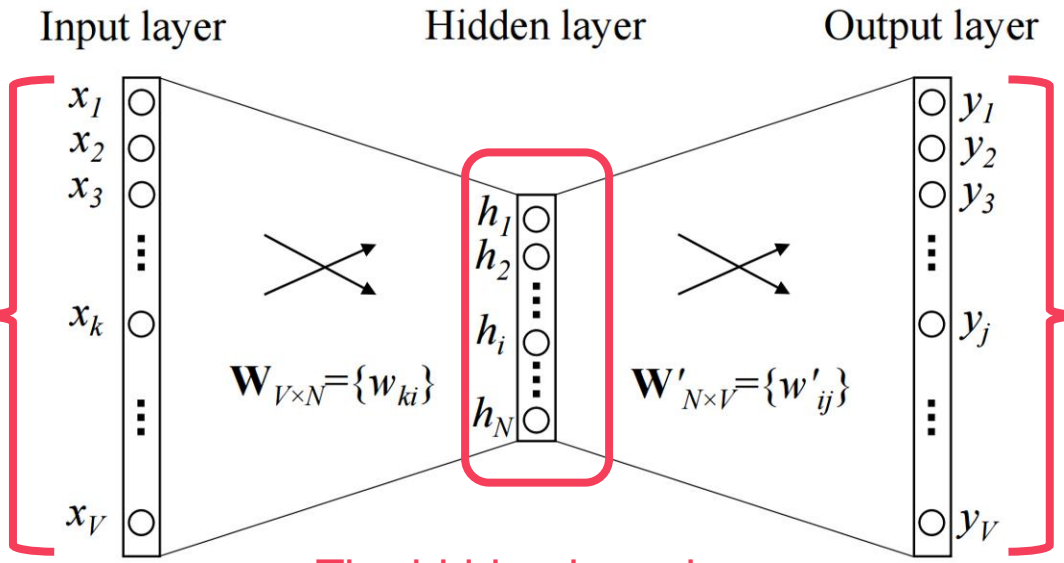
The input is one-hot vector of size V

The output is a length V vector with the elements being the softmax values

The hidden layer is the embedding layer

Word2Vec: Architecture

- Let V = size of vocabulary and N = embedding dimension
- Two different weight matrices:
  - $W_{V\times N}$: from input to hidden layer
  - $W'_{N\times V}$: from hidden to output layer



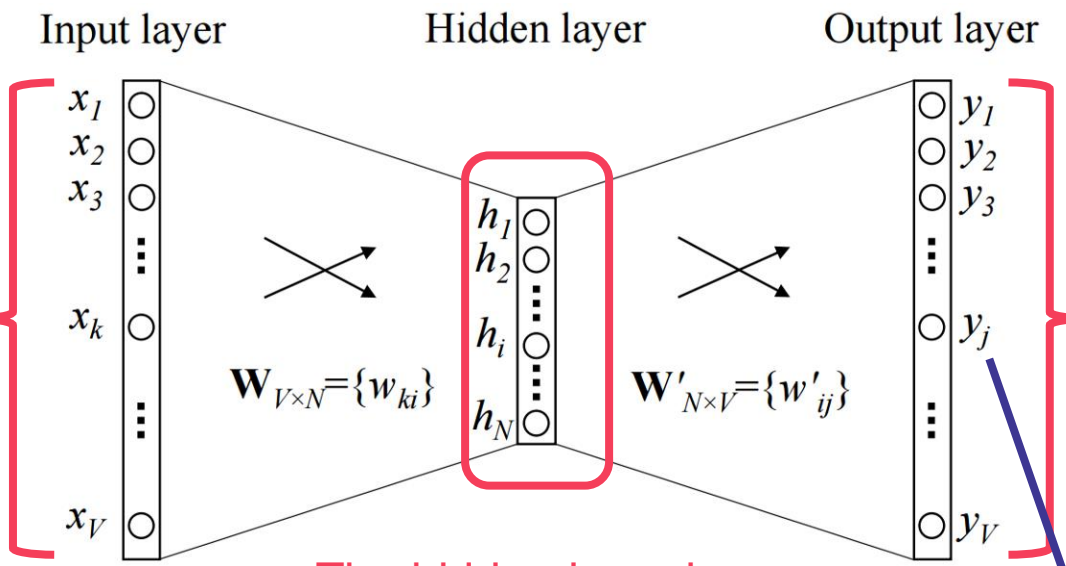
The input is one-hot vector of size V

The output is a length V vector with the elements being the softmax values

The hidden layer is the embedding layer

Word2Vec: Architecture

- Let V = size of vocabulary and N = embedding dimension
- A softmax function is applied on output layer to convert output to probability distribution



The input is one-hot vector of size V

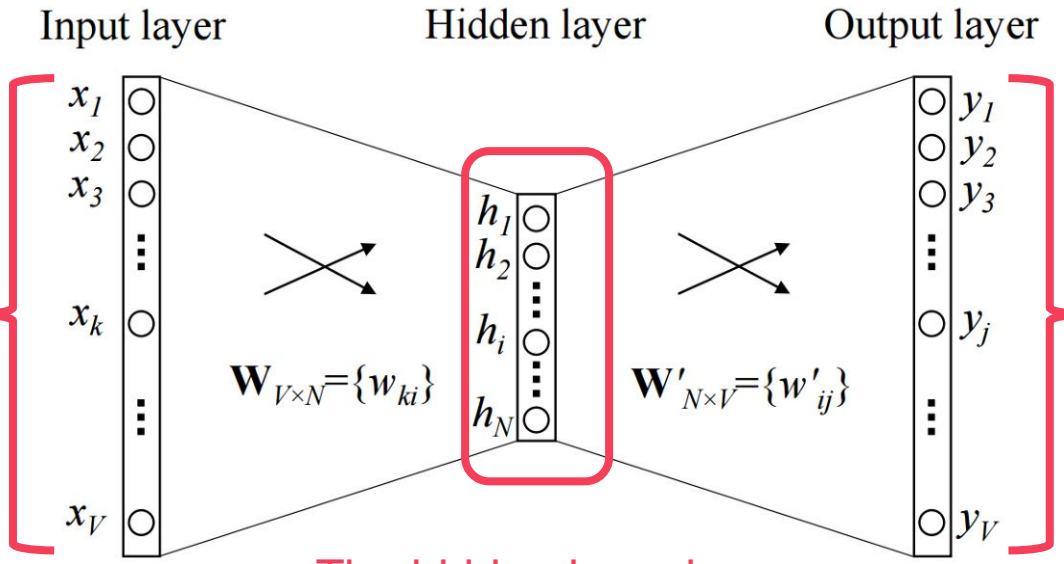
The output is a length V vector with the elements being the softmax values

The hidden layer is the embedding layer

Probability that j-th word is in the context

Word2Vec: Architecture

- Let V = size of vocabulary and N = embedding dimension
- The hidden layer is all linear neurons, no activation!
- After training the network, embedding of a word is obtained by $x^T W$
  - i.e., matrix multiplication between word's one-hot vector and learned weights $W_{V \times N}$



The input is one-hot vector of size V

The output is a length V vector with the elements being the softmax values

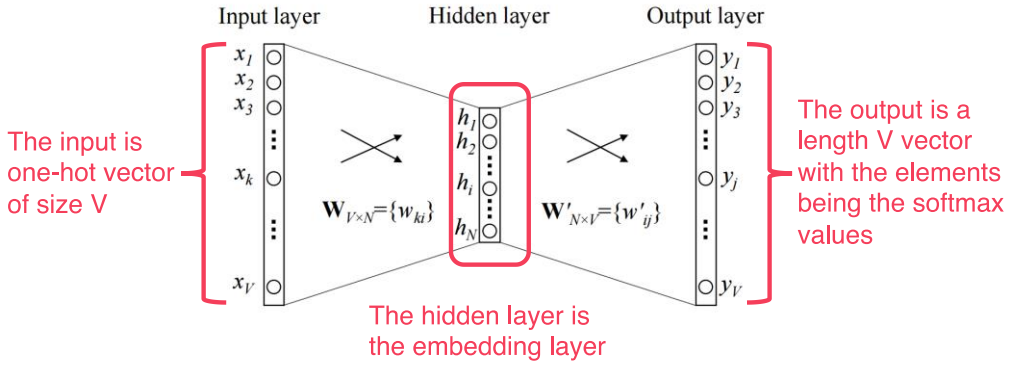
The hidden layer is the embedding layer

Word2Vec: Training the Network

- How is the network trained?
  - There are no labels → It is an unsupervised task (it is a statistical method based on co-occurrence of words in one window)
  - We therefore create a fake task!

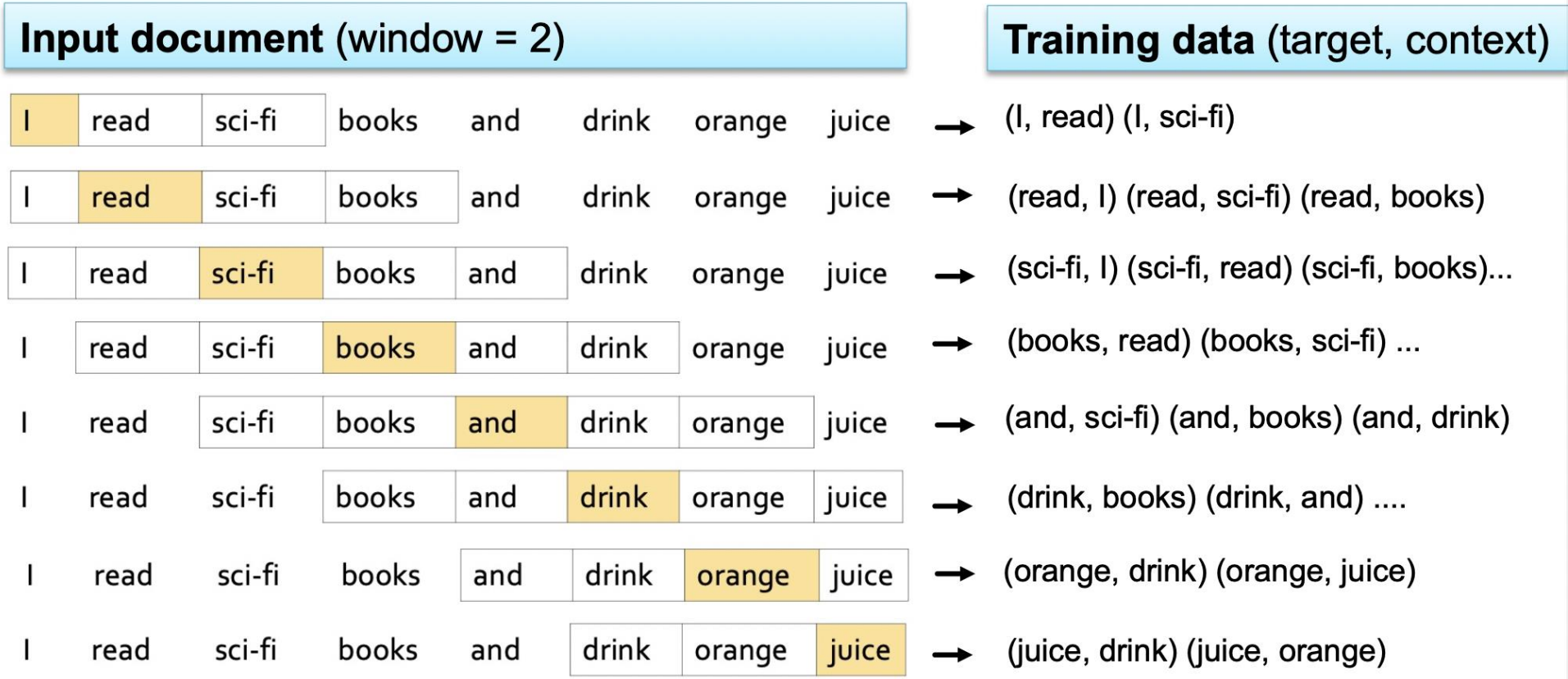
- Fake task = given a target word, predict its context words

- Decisions to make:
  - How to make training data?
  - What is the loss function?



Word2Vec: Training Data

- Ex: Document = {I read sci-fi books and drink orange juice}



Word2Vec: Loss Function

- Given the topology of the network, if x is the input and y is the output, then

$$output = y = softmax(W'^T W^T x)$$

- We train against target-context pairs (w_t, w_c)
  - The context word w_c represents the ideal prediction, given the target word w_t

- w_c is represented as one-hot (i.e., it has value 1 at some position j and other positions are 0)

Word2Vec: Loss Function

- The loss function needs to evaluate the output layer at the same position j, i.e., y_j
  - remember y is a probability distribution; ideal value of y_j is being 1
- We use cross-entropy loss function
- Given two probability distributions ***p*** and ***q***, it is defined:

$$\mathrm{CE}(w_c, y) = -\log(y_{correct_class})$$

position j

If w_c = [0,0,0,...,1,0,0,0..,0] and y = [0.02, 0.11,.... 0.8, 0, 0.031, ...]
then the loss value would be

$$L = -\log(0.8)$$

Word2Vec: Backpropagation

- Now that loss function is clear, we want to find the values of ***W*** and ***W'*** that minimize it
  - We want our model to learn the weights

- We use gradient descent to tackle this
  - Find derivatives $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial W'}$
  - Update weights as $W_{new} = W_{old} - \mu \frac{\partial L}{\partial W}$

Word2Vec: Example

- Document = {I read sci-fi books and drink orange juice}
  - Since this is only document in our corpus, our vocab is:

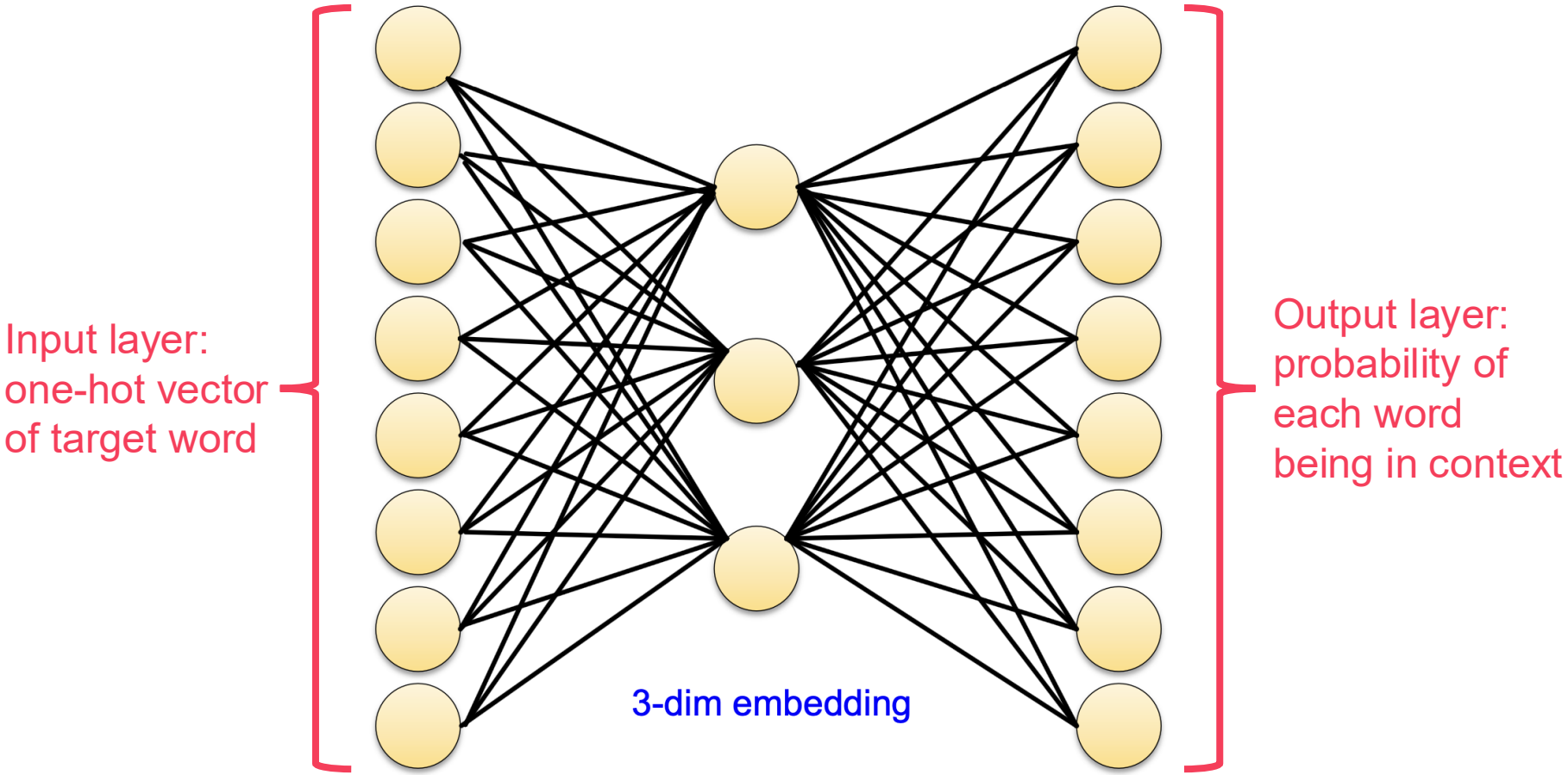
vocab = [“I”, “read”, ”sci-fi”, ”books”, “and”, “drink”, “orange”, “juice”] and V = 8

- We execute one forward pass using above document
- First of all, assign one-hot vectors to words

I:	[1,	0,	0,	0,	0,	0,	0,	0]
read:	[0,	1,	0,	0,	0,	0,	0,	0]
sci-fi:	[0,	0,	1,	0,	0,	0,	0,	0]
books:	[0,	0,	0,	1,	0,	0,	0,	0]
and:	[0,	0,	0,	0,	1,	0,	0,	0]
drink:	[0,	0,	0,	0,	0,	1,	0,	0]
orange:	[0,	0,	0,	0,	0,	0,	1,	0]
juice:	[0,	0,	0,	0,	0,	0,	0,	1]

Word2Vec: Example

- Size of vocabulary(V) = 8 and let's set embedding dim = 3



Word2Vec: Example

- If target word = books and weight matrix $W_{V\times N}$ be as following:

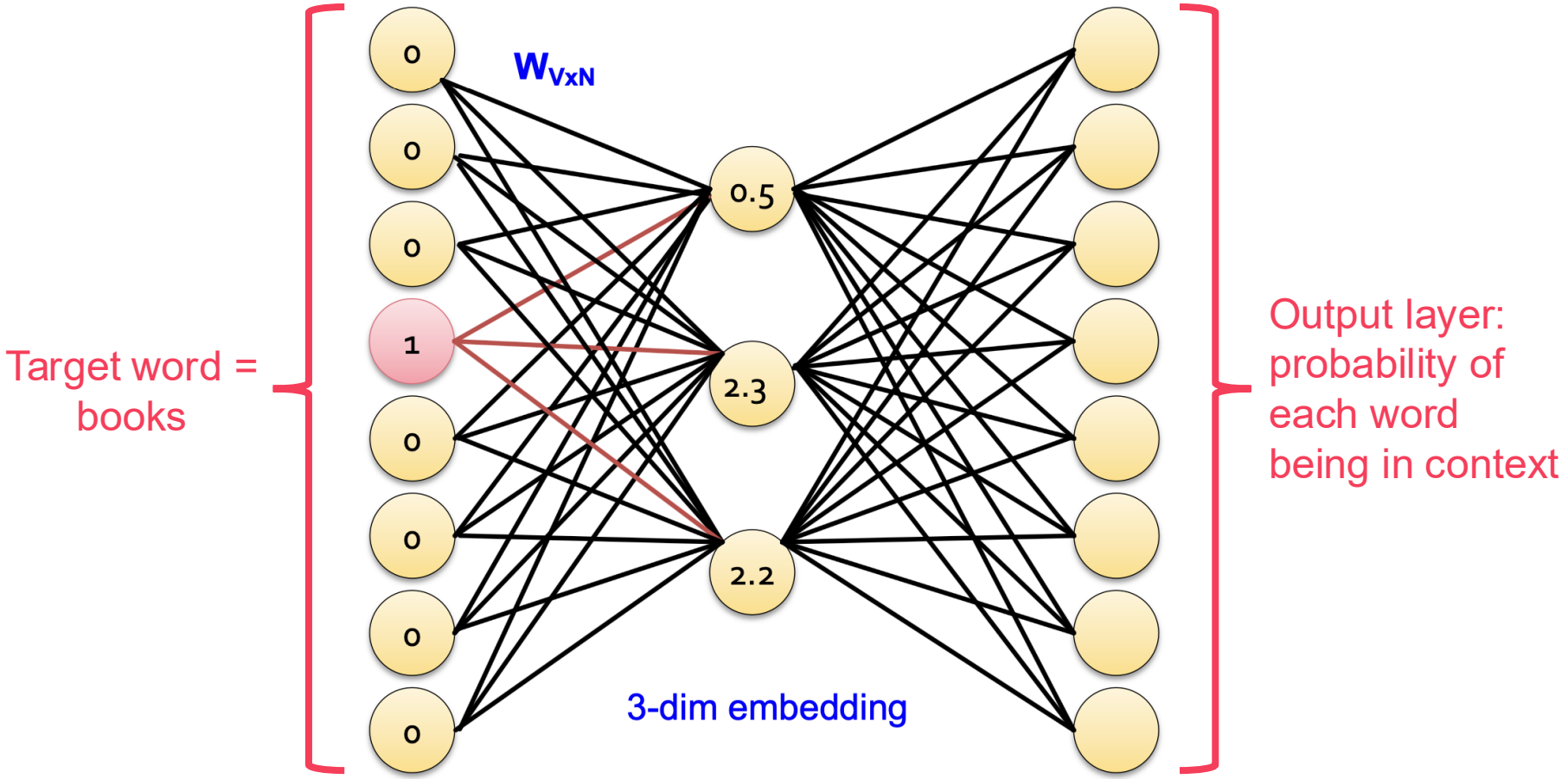
$$W_{V\times N} = \begin{bmatrix} 1 & 2 & 2 \\ -1.2 & -3 & -2 \\ 1.2 & 1.1 & 0.5 \\ 0.5 & 2.3 & 2 \\ -1.1 & 0.6 & -1 \\ 1 & -1 & 2 \\ 0.3 & 1.2 & 0.7 \end{bmatrix}$$

- Then

$$[0,0,0,1,0,0,0,0]\times \begin{bmatrix} 1 & 2 & 2 \\ -1.2 & -3 & -2 \\ 1.2 & 1.1 & 0.5 \\ 0.5 & 2.3 & 2 \\ -1.1 & 0.6 & -1 \\ 1 & -1 & 2 \\ 0.3 & 1.2 & 0.7 \end{bmatrix} = [0.5, 2.3, 2.2]$$

Word2Vec: Example

- If target word = books, and $W_{V \times N}$ given:



Word2Vec: Example

- If the weight matrix $W'_{N\times V}$ be as following:

$$W'_{N\times V} = \begin{bmatrix} 1 & 2 & 2 & 0 & 0.7 & 1.3 & -1 & -0.1 \\ 1.2 & 0.5 & -1 & 1 & 0.3 & 2 & .6 & 1 \\ -1 & 1.6 & -0.5 & 1.4 & 2.3 & 1 & 1 & 0.6 \end{bmatrix}$$

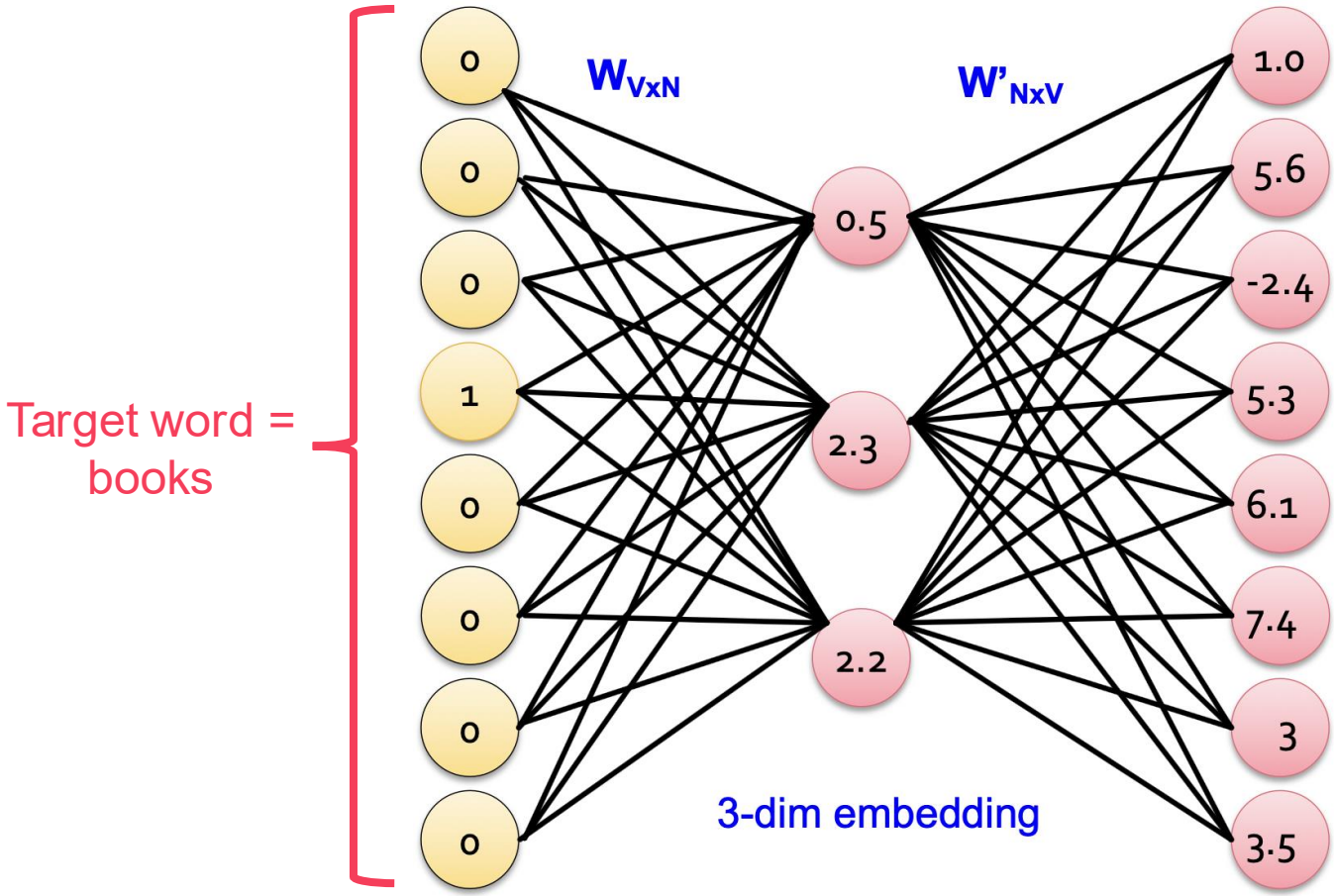
- Then

$$[0.5,2.3,2.2]\times \begin{bmatrix} 1 & 2 & 2 & 0 & 0.7 & 1.3 & -1 & -0.1 \\ 1.2 & 0.5 & -1 & 1 & 0.3 & 2 & 0.6 & 1 \\ -1 & 1.6 & -0.5 & 1.4 & 2.3 & 1 & 1 & 0.6 \end{bmatrix}$$

= [1.0, 5.6, -2.4, 5.3, 6.1, 7.4, 3.0, 3,5]

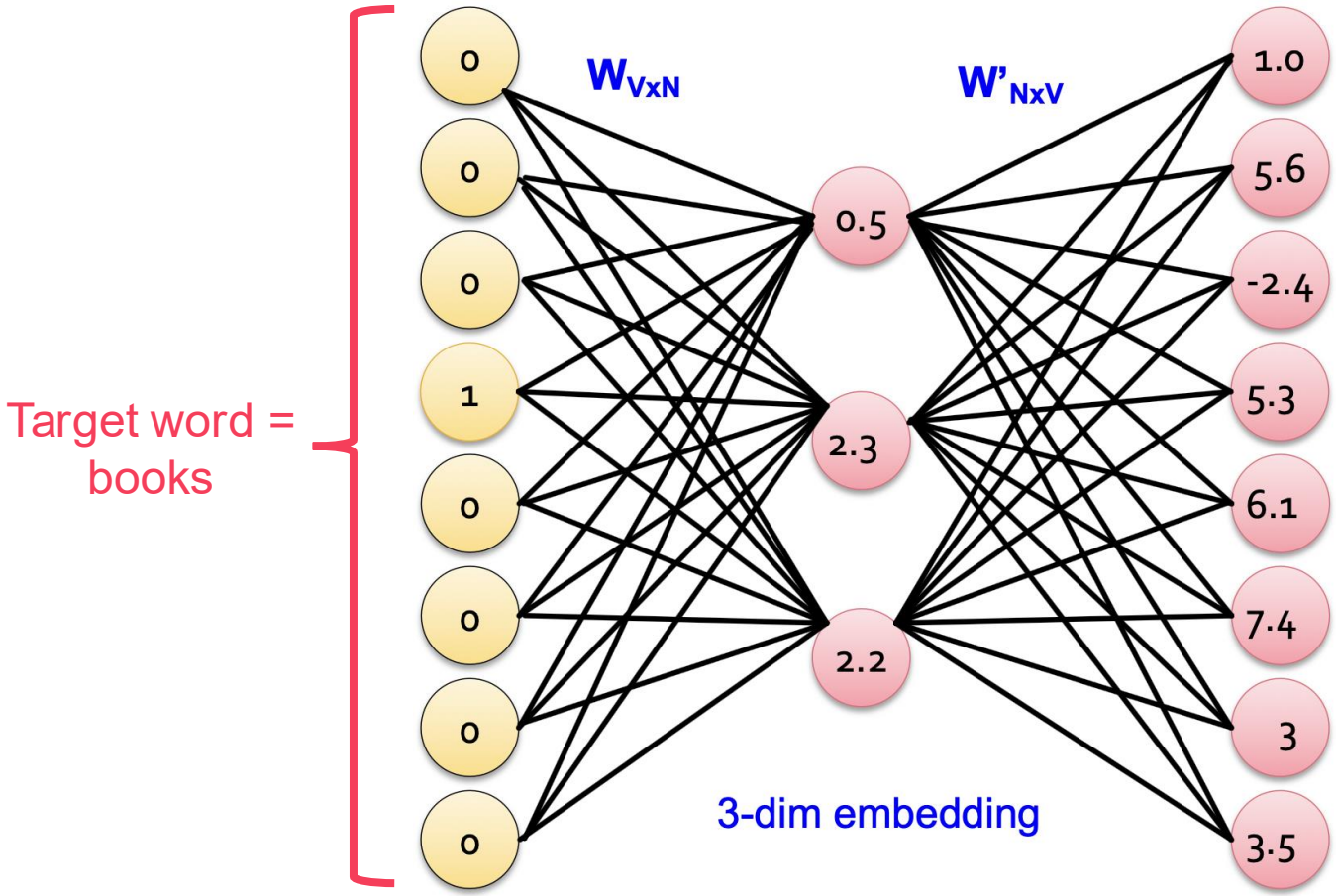
Word2Vec: Example

- If $W'_{N\times V}$ is given:



Word2Vec: Example

- If $W'_{N\times V}$ is given:

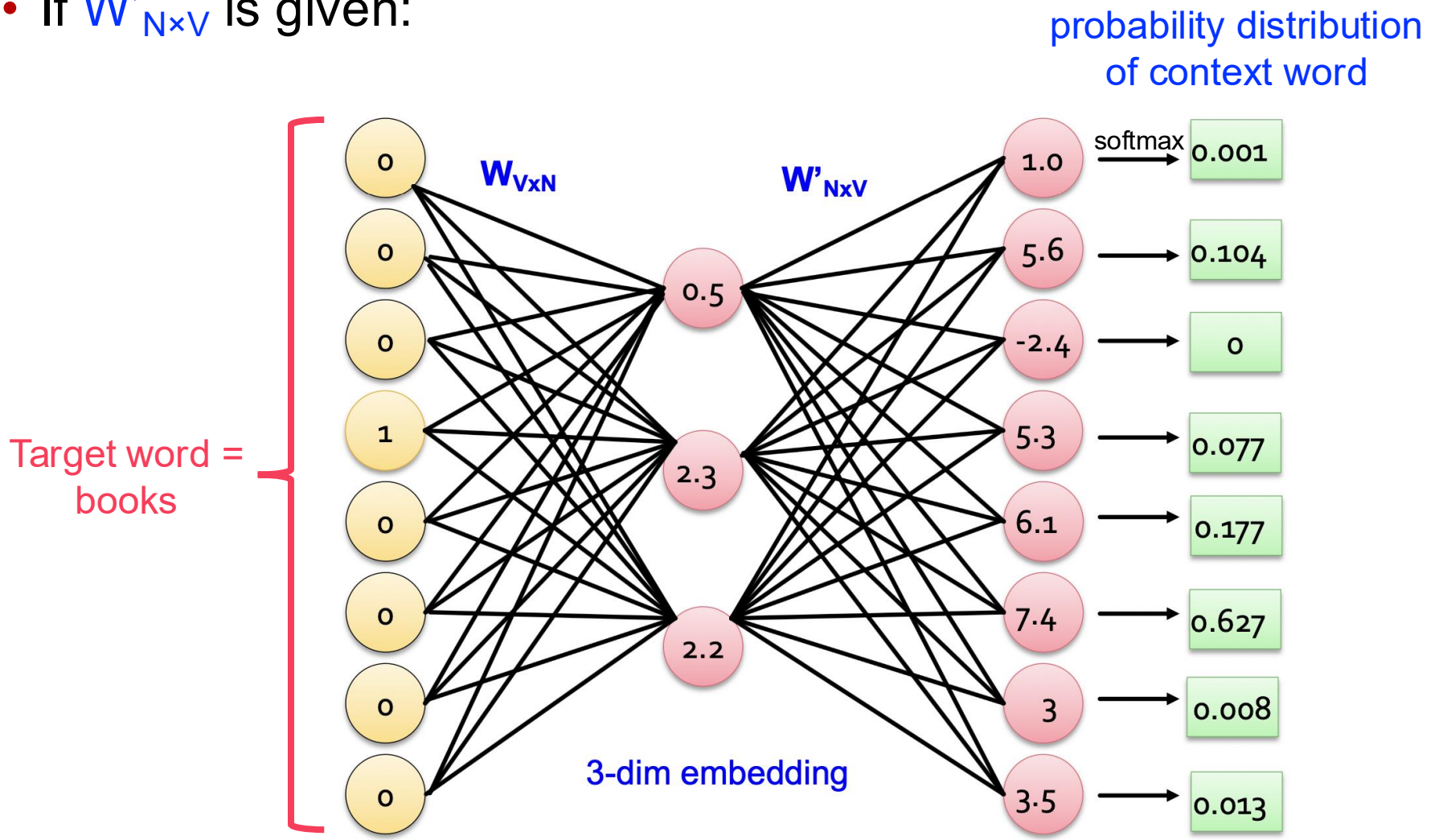


We apply softmax functions to turn them into probabilities

Each output node x = $\frac{e^x}{\sum e^x}$

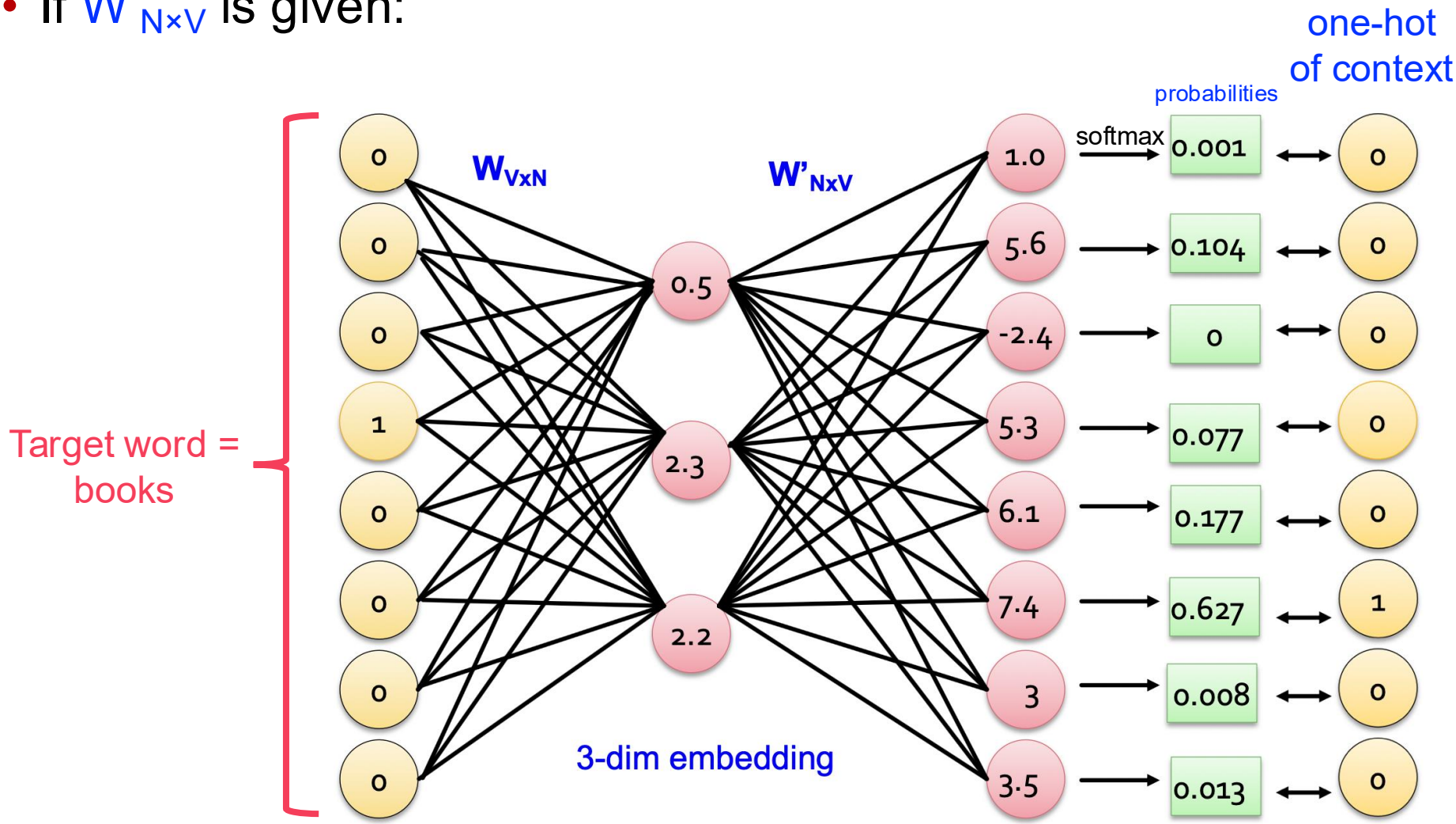
Word2Vec: Example

- If $W'_{N\times V}$ is given:



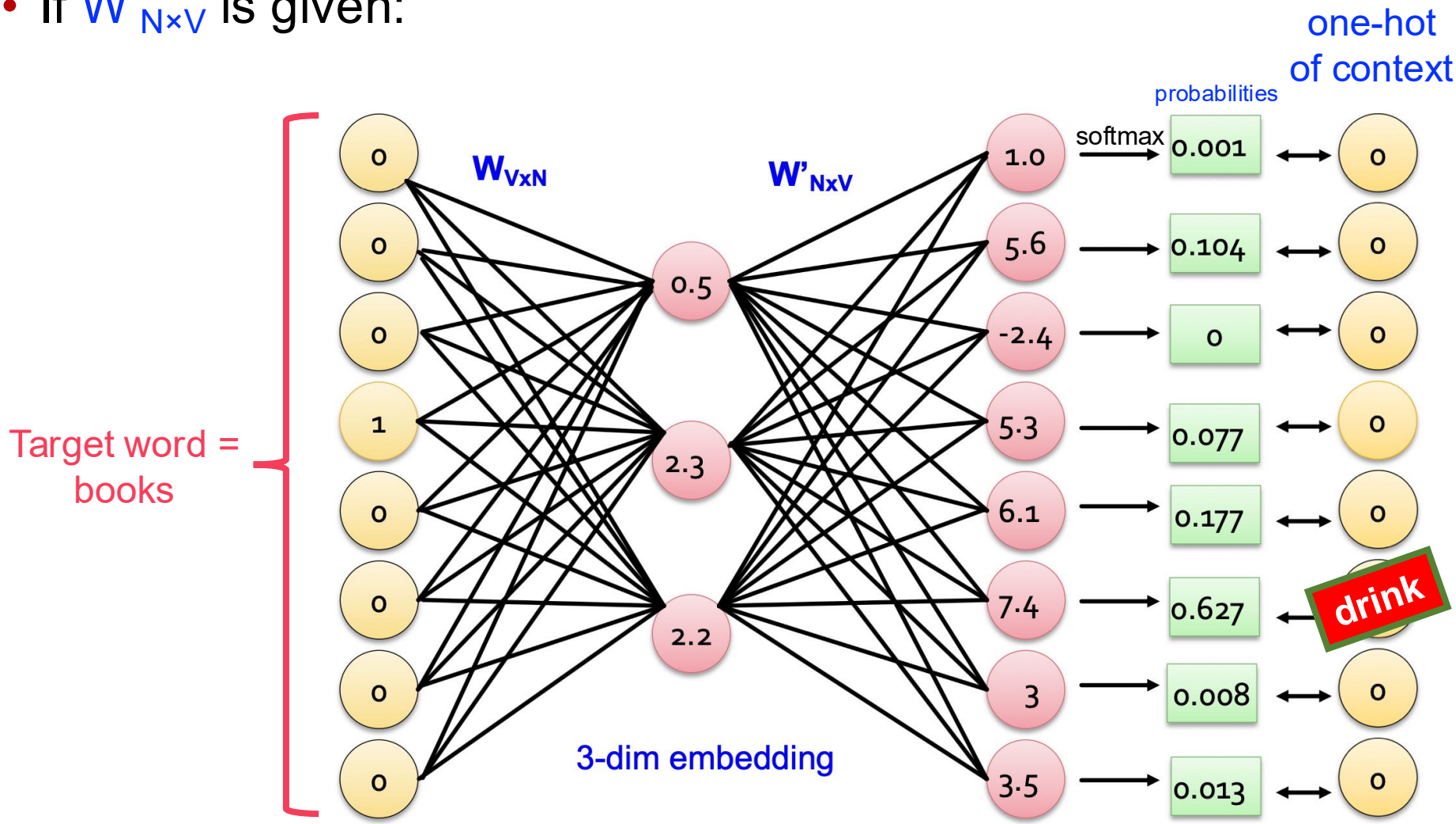
Word2Vec: Example

- If $W'_{N\times V}$ is given:



Word2Vec: Example

- If $W'_{N\times V}$ is given:



Word2Vec: Example

- The network learns by comparing softmax vector to the one-hot of true context word

- In our example target = “books”, one correct context =“read” but we predicted “drink”
  - Predicted vector = [0.001,0.104,0,0.077,0.177,0.627,0.008,0.013]
  - One-hot of “read”= [0,1,0,0,0,0,0,0]
  - Then the loss value would be

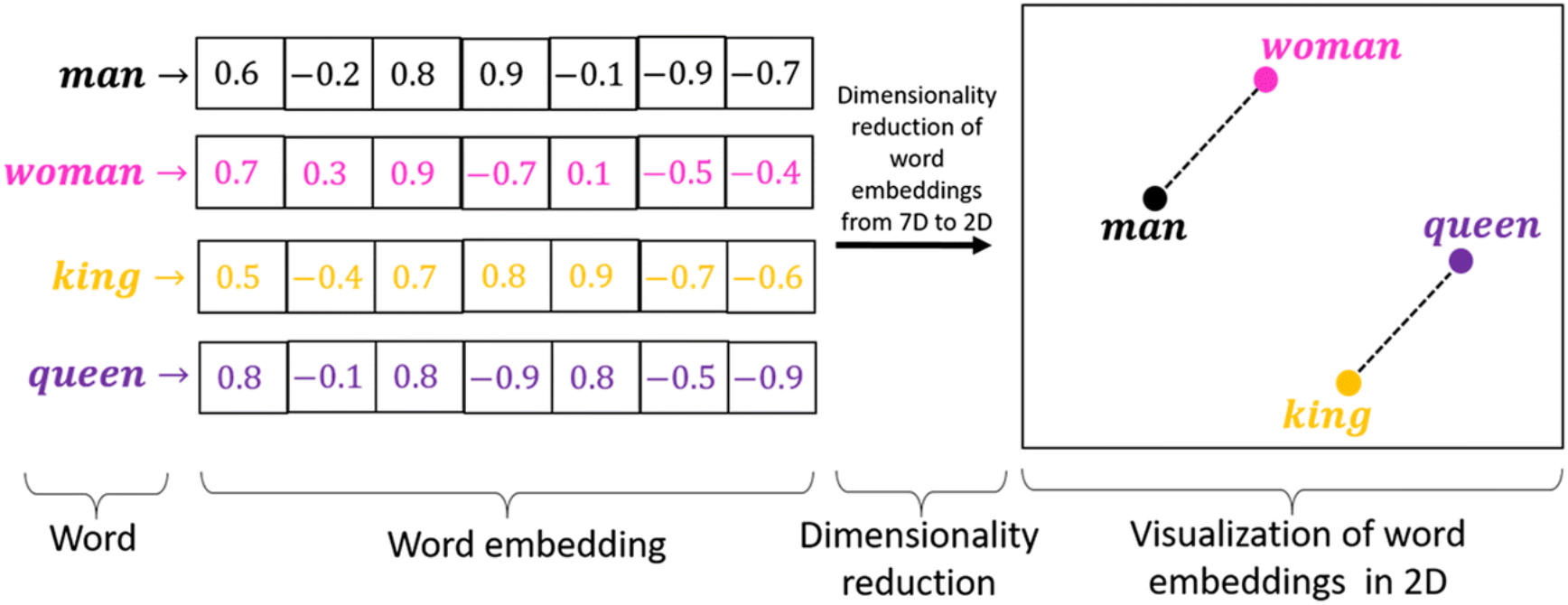
$$L = -\ln(0.104) = 2.26$$

Word2Vec: Summary

- Word2Vec comes in two architecture
  - CBOW: Given context words, it predicts target word
  - Skip-Gram: Given target word, predicts context words
- Skip Gram method
  - works well with small amount of data and is found to represent rare words well
- CBOW method
  - is faster and more suitable for large data, it has better representations for more frequent words

Word2Vec: Summary

- Word2vec assigns an embedding to every word in the vocabulary
- Embedding dimension << size of the vocabulary





Learning Embeddings – part 2

Task Independent vs. Task Specific

- So far we worked with unsupervised data
  - A corpus of documents
- We learned embeddings that was not tied to any classification or regression task
  - We created a fake task of predicting nearby words
- Alternatively, we can learn embeddings for a specific tasks such as classification

Embedding Layer

- We can learn an embedding as part of a neural network for a given task, e.g., prediction task
  - Pros: produces an embedding customized for a particular task
  - Cons: takes longer than training the embedding separately

- Embedding layer is a regular hidden layer of size *d* (i.e, *d* hidden units)

- We combine the embedding layer with any other features needed for our specific task

Example: Recommending Movies

- **Input**: 1 million movies, and 500k users who have watched some of these movies
- **Task**: recommend movies to users

- We solved this problem before using collaborative filtering, and latent factor models

- Here, we formulate it as multi-class classification where each movie is a class
- We use neural network to learn embeddings for movies such that similar movies have similar embeddings

Example: Recommending Movies

- **Train/Test split**: First split data into train and test
  - For every user, randomly hold out few movies they have watched as test and use the rest to build train data

Full data
Alice -> m1, m2, m3, m4, m5
Bob -> m8, m9, m21
Sam -> m2, m6, m10

split →

Train	Test
Alice -> m1, m4, m5	Alice -> m3, m2
Bob -> m8, m9	Bob -> m21
Sam -> m6, m10	Sam -> m2

Example: Recommending Movies

- **Build train data**: We then build train data as pairs (movie1, movie2) where both movies are watched by same user

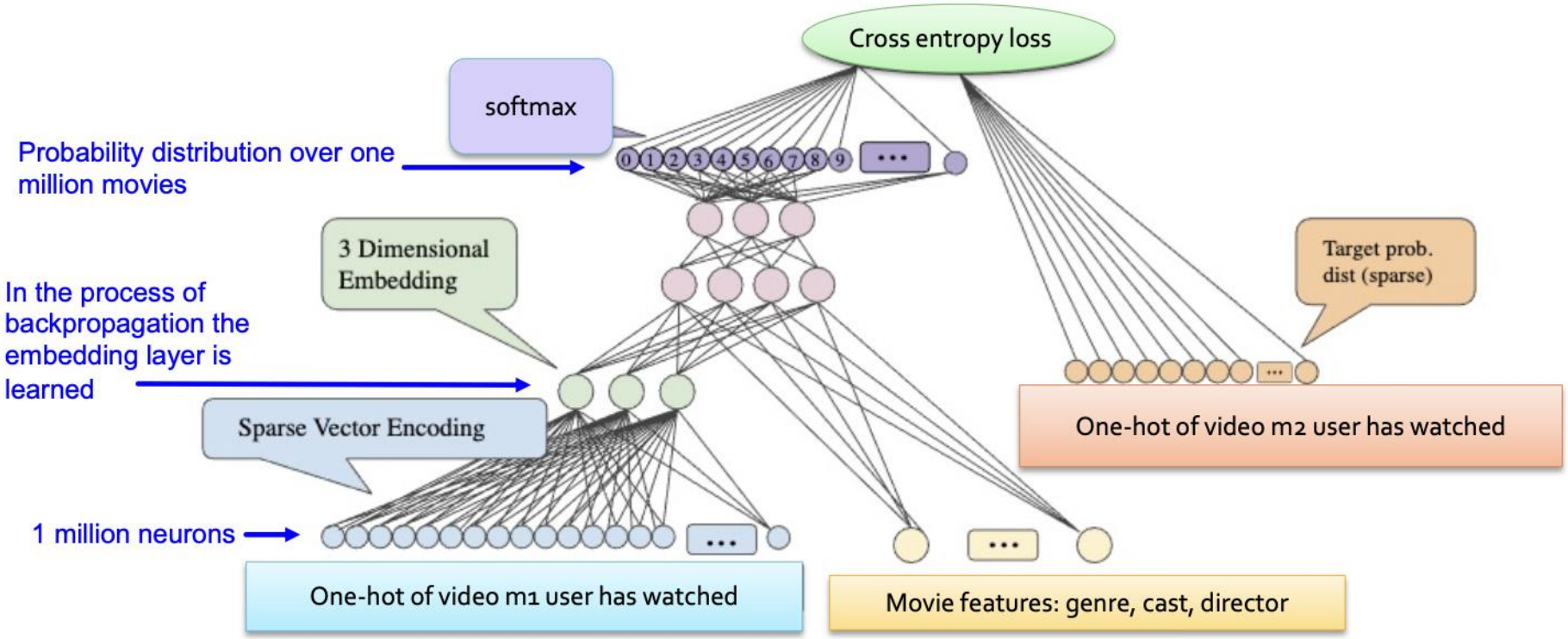
Train	Test
Alice -> m1, m4, m5	Alice -> m3, m2
Bob -> m8, m9	Bob -> m21
Sam -> m6, m10	Sam -> m2

Prepare train data for NN

Train	Test
(m1, m4) (m4, m5) ...	m3, m2
(m8, m9) (m9, m8)	m21
(m6, m10) (m10, m6)	m2

Example: Recommending Movies

- We then build a neural network that performs collaborative filtering while learning 3-dimensional embeddings



Example: Recommending Movies

- How to recommend a movie to a user e.g. Alice?
  - Alice has watched m1, m4, m5 in train
  - Find movies that have similar embeddings to m1
    - Similarity score = <emb(m1), emb(v)> for any movie v
    - Find top 5 movies with highest similarity score
    - Recommend them to Alice

- Or even better: repeat above for m1, m4 and m5
  - Recommend movies in intersection of above sets

So far…

- So far we have seen examples of converting one-hot encodings to embeddings
  - Word2Vec
  - Supervised NN with one-hot input vector
- We can use NN to learn embedding from dense feature vectors
  - What other method does the same? SVD, NMF, PCA

Autoencoder

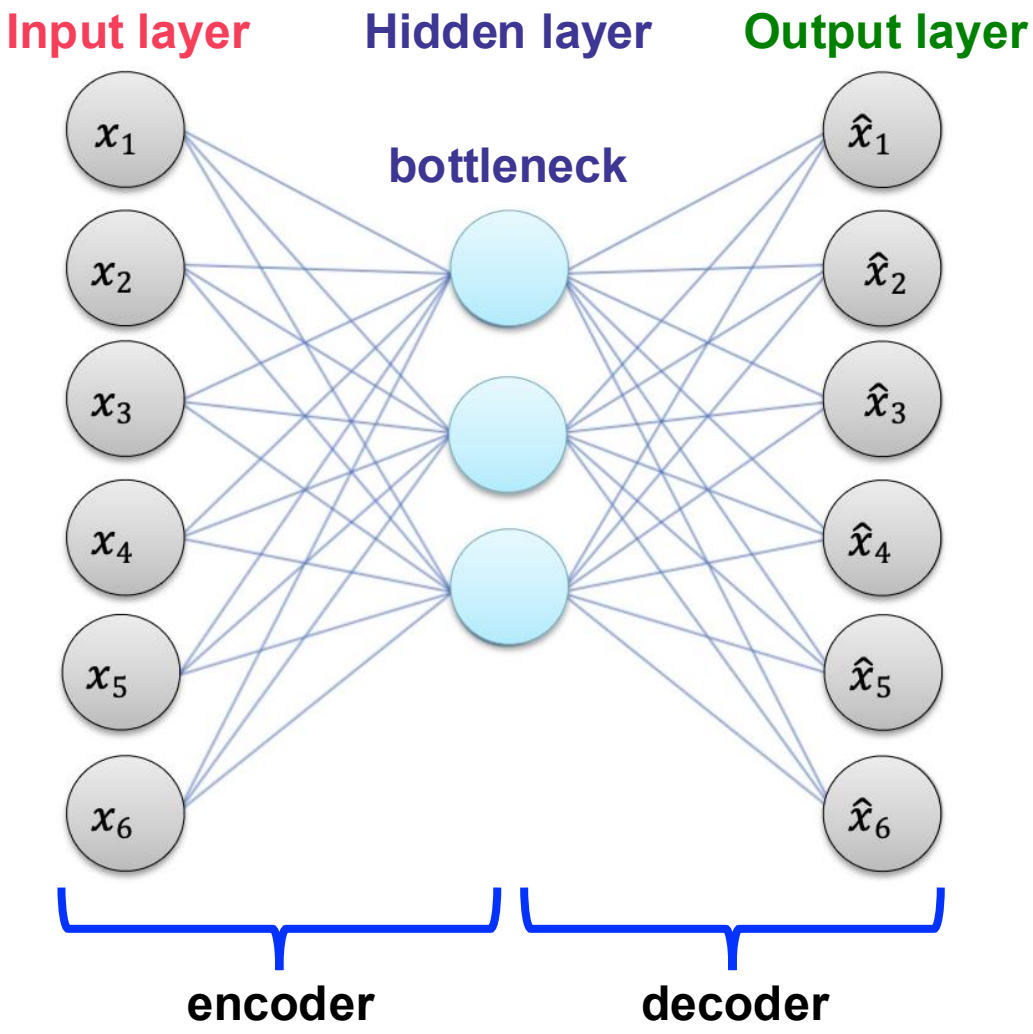
- Autoencoder are an extension of PCA to non-linear space
- They are a special type of neural network that is trained to copy input to output except that it has to go through a bottleneck
  - They are unsupervised too
- It learns to compress the data while minimizing the reconstruction error

Autoencoder

Input layer: is input feature vector. It does not need to be one-hot vectors. Here, input data is 6-dim vectors

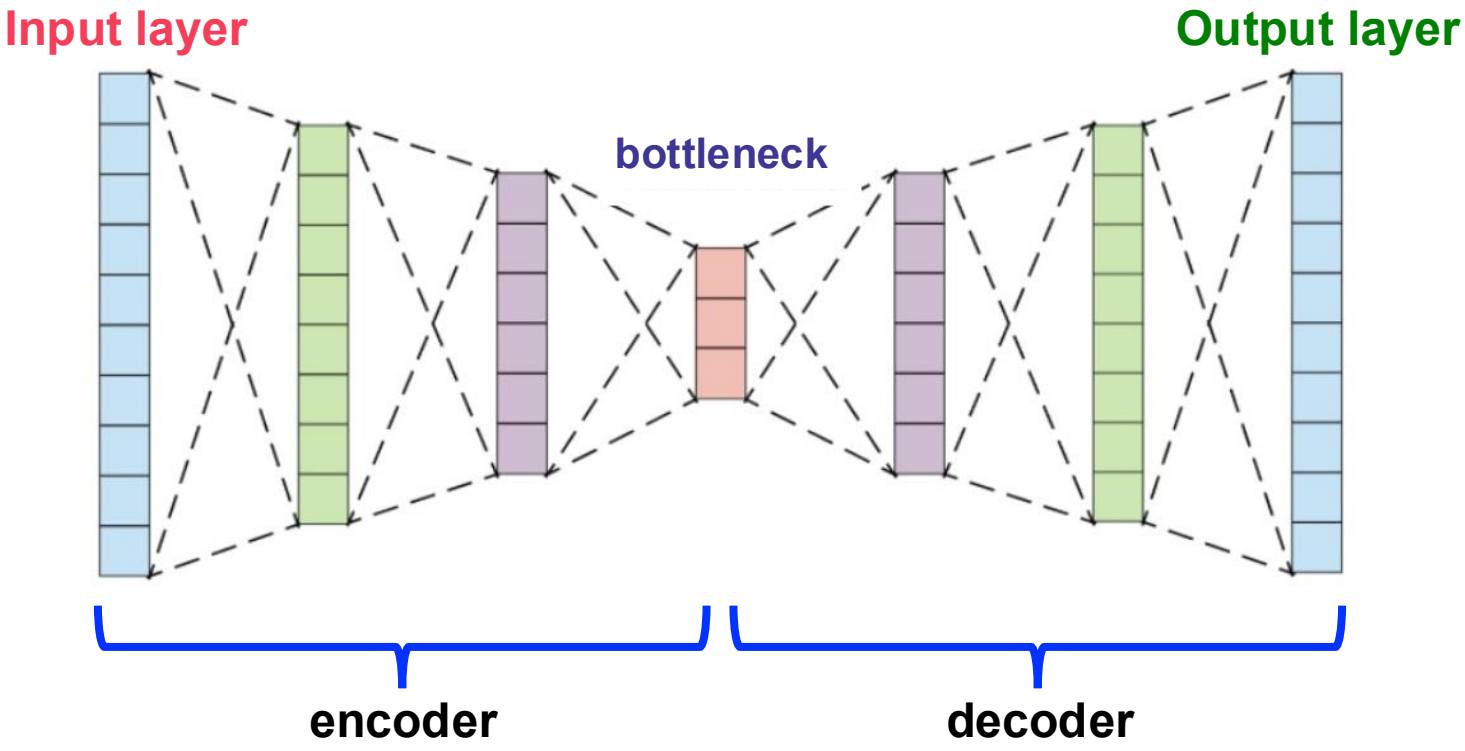
Bottleneck layer: is the bottleneck as it projects down 6-dim vector to 3-dim space. It constrains the amount of information that traverses the network

Output layer: is the reconstructed input from 3-dim to 6-dim



Autoencoder

- There can be multiple hidden layers between input layer and bottleneck layer, similarly between bottleneck layer and output layer



Autoencoder

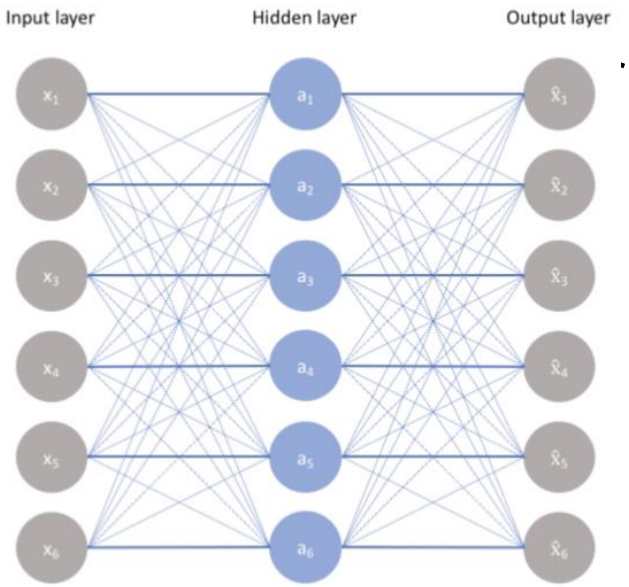
- Two main component in their architecture:

- **Encoder**: a function **f** that compresses the input into a latent-space representation
  - f(x) = h such that dimension(h) < dimension(x)

- **Decoder**: a function **g** that reconstruct the input from the latent space representation
  - g(h) ~ x, i.e., bring h back to the original space

Autoencoder

- The bottleneck is the key:
  - Without an information bottleneck, could learn to memorize



- There are different types of autoencoders:
  - Undercomplete, denoising, sparse, variational
  - Today, we talk about undercomplete autoencoder
    - i.e., bottleneck dimension < input dimension

Autoencoder: Training

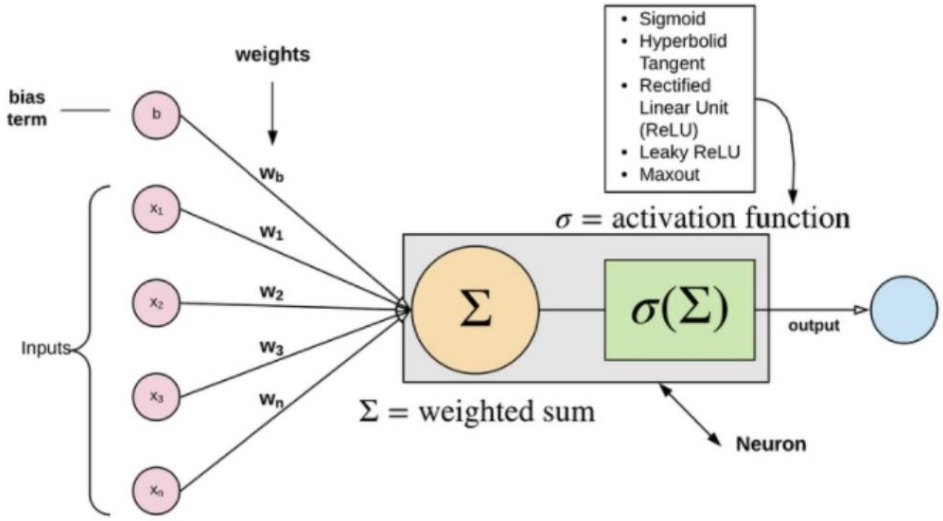
- The loss function to train an undercomplete autoencoder is *reconstruction loss*:

$$L(x, \hat{x}) = ||x - \hat{x}||_1$$

- No regularization term is needed in undercomplete autoencoder to ensure the model is not memorizing the input data we regulate:
  - size of the bottleneck layer
  - number of hidden layers

Autoencoder is a non-linear PCA

- **A neuron has activation functions**. As long as activation function is not identity, we learn non-linear embedding



- If we use identity activation functions in hidden layers, we convert back to PCA and produce similar dimensionality reduction as PCA

Autoencoder: Summary

- Non-linear PCA
- A neural network that is trained to copy input to output
  - It passes data through a bottleneck
  - Reconstruction loss function: L1, KL divergence
  - Unsupervised
- There are different types of autoencoders:
  - Undercomplete, denoising, sparse, variational
  - We studied undercomplete autoencoder

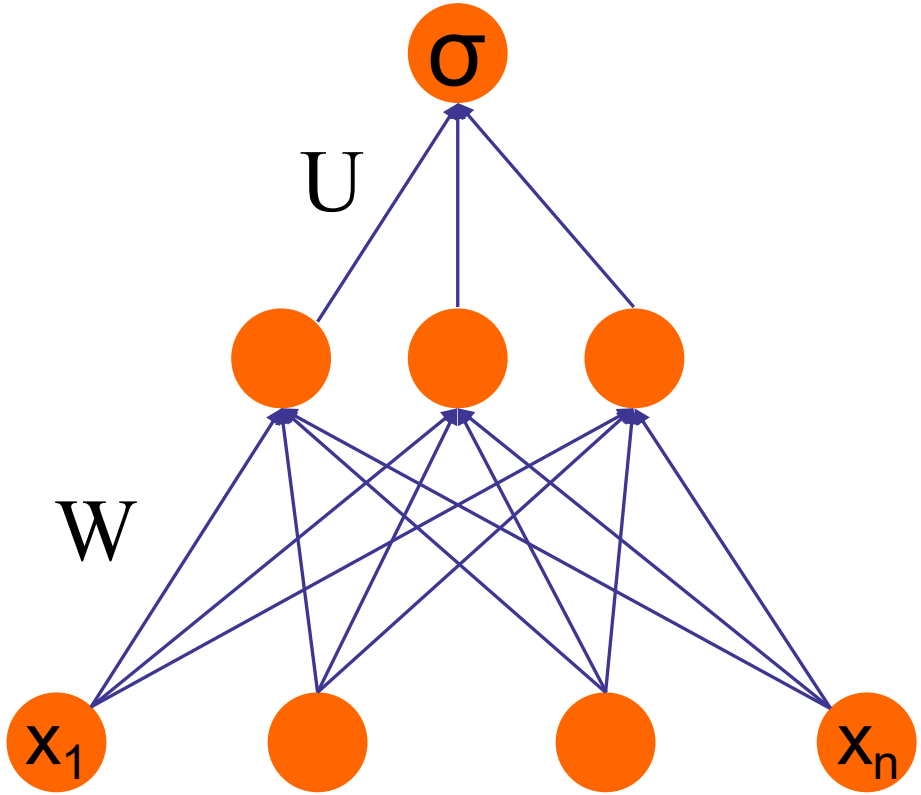
Use Cases for Embeddings with Feedforward Networks

- Let's consider 2 (simplified) sample tasks:
  1. Text classification
  2. Language modeling

- State of the art systems use more powerful neural architectures, but simple models are useful to consider!

Classification: Sentiment Analysis

- We could do exactly what we did with logistic regression
- Input layer are binary features as before
- Output layer is 0 or 1

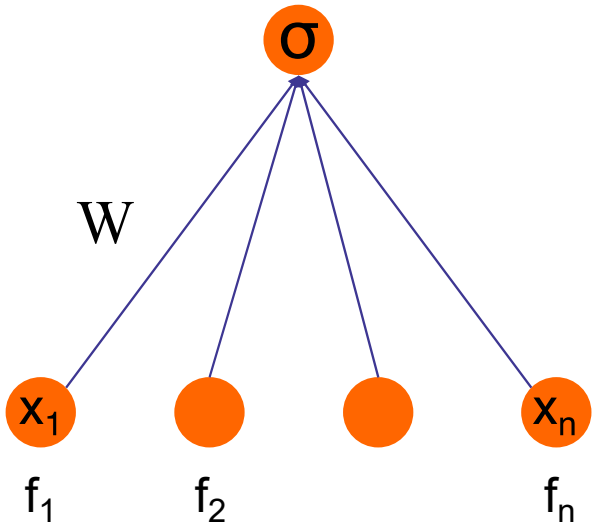


Sentiment Features

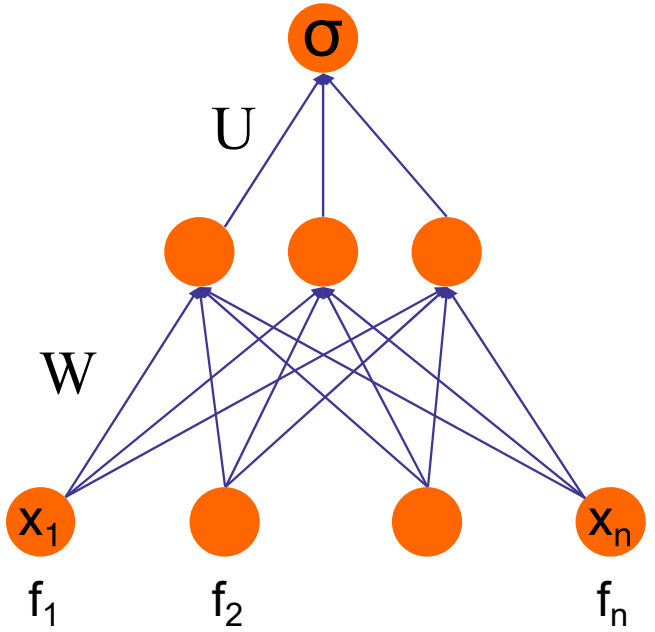
Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

Feedforward Networks for Simple Classification

Logistic Regression



2-layer feedforward network

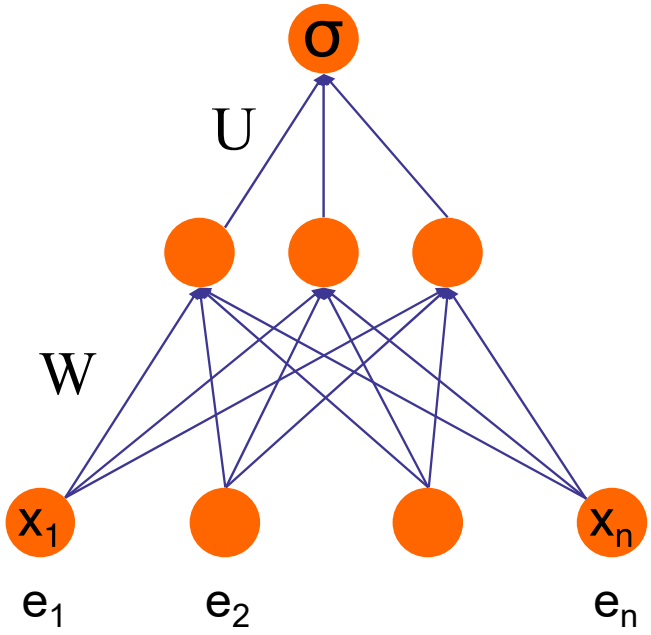


- Just adding a hidden layer to logistic regression
  - allows the network to use non-linear interactions between features
  - which may (or may not) improve performance

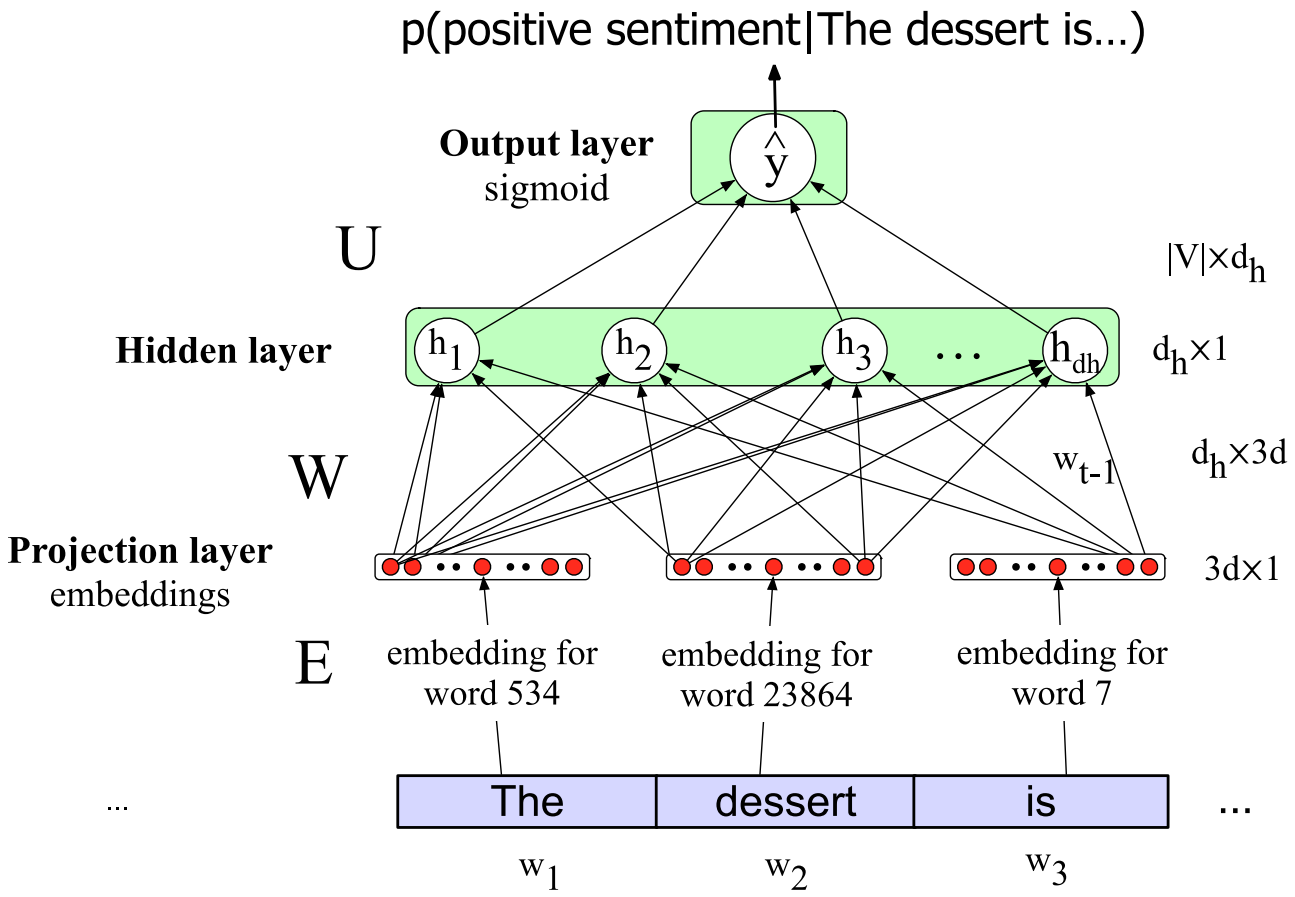
Even Better: Representation Learning

- The real power of deep learning comes from the ability to **learn** features from the data

- Instead of using hand-built human-engineered features for classification, use learned representations like **embeddings**!

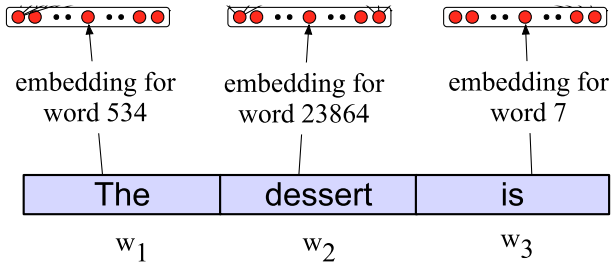


Neural Network Classification with Embeddings as Input Features!



Issue: Texts Come in Different Sizes

- This assumes a fixed size length (3)!
- Kind of unrealistic

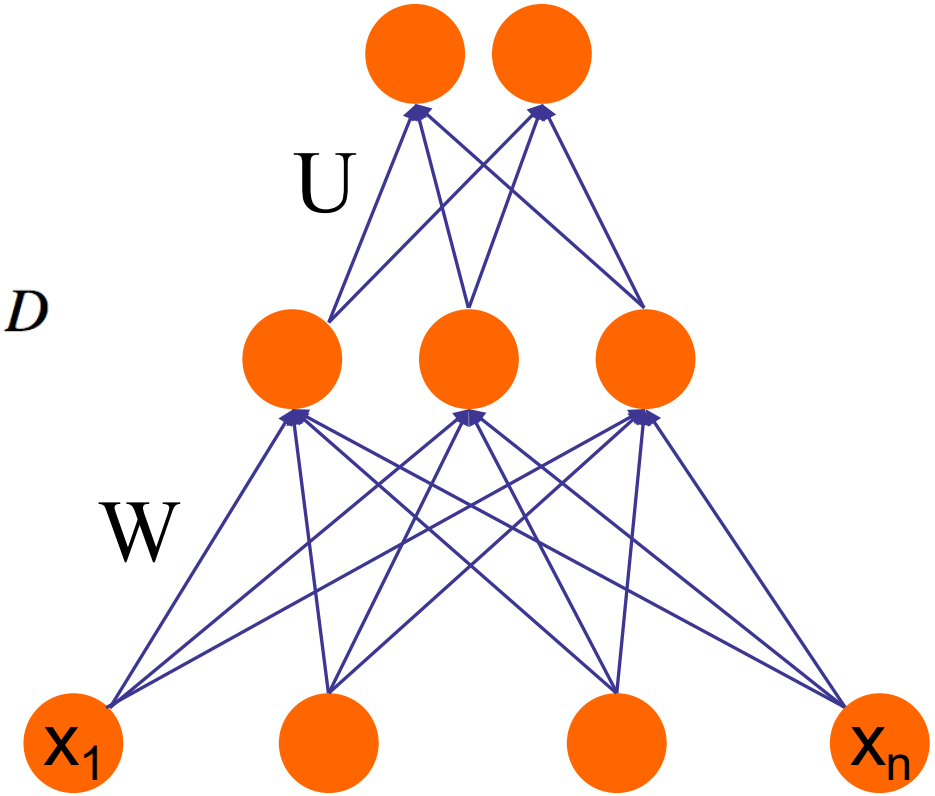


- Some simple solutions (more sophisticated solutions later)
  1. Make the input the length of the longest review
     - If shorter then pad with zero embeddings
     - Truncate if you get longer reviews at test time
  2. Create a single "sentence embedding" (the same dimensionality as a word) to represent all the words
     - Take the mean of all the word embeddings
     - Take the element-wise max of all the word embeddings
       - For each dimension, pick the max value from all words

Reminder: Multiclass Outputs

- What if you have more than two output classes?
  - Add more output units (one for each class)
  - And use a “softmax layer”

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \quad 1 \leq i \leq D$$



Neural Language Models (LMs)

- **Language Modeling**: Calculating the probability of the next word in a sequence given some history
  - Neural network based LMs far outperform N-gram based LMs
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers
- But **simple feedforward LMs** can do almost as well!

Simple feedforward Neural Language Models

- **Task**:

  predict next word w_t given prior words w_{t-1}, w_{t-2}, w_{t-3}, …

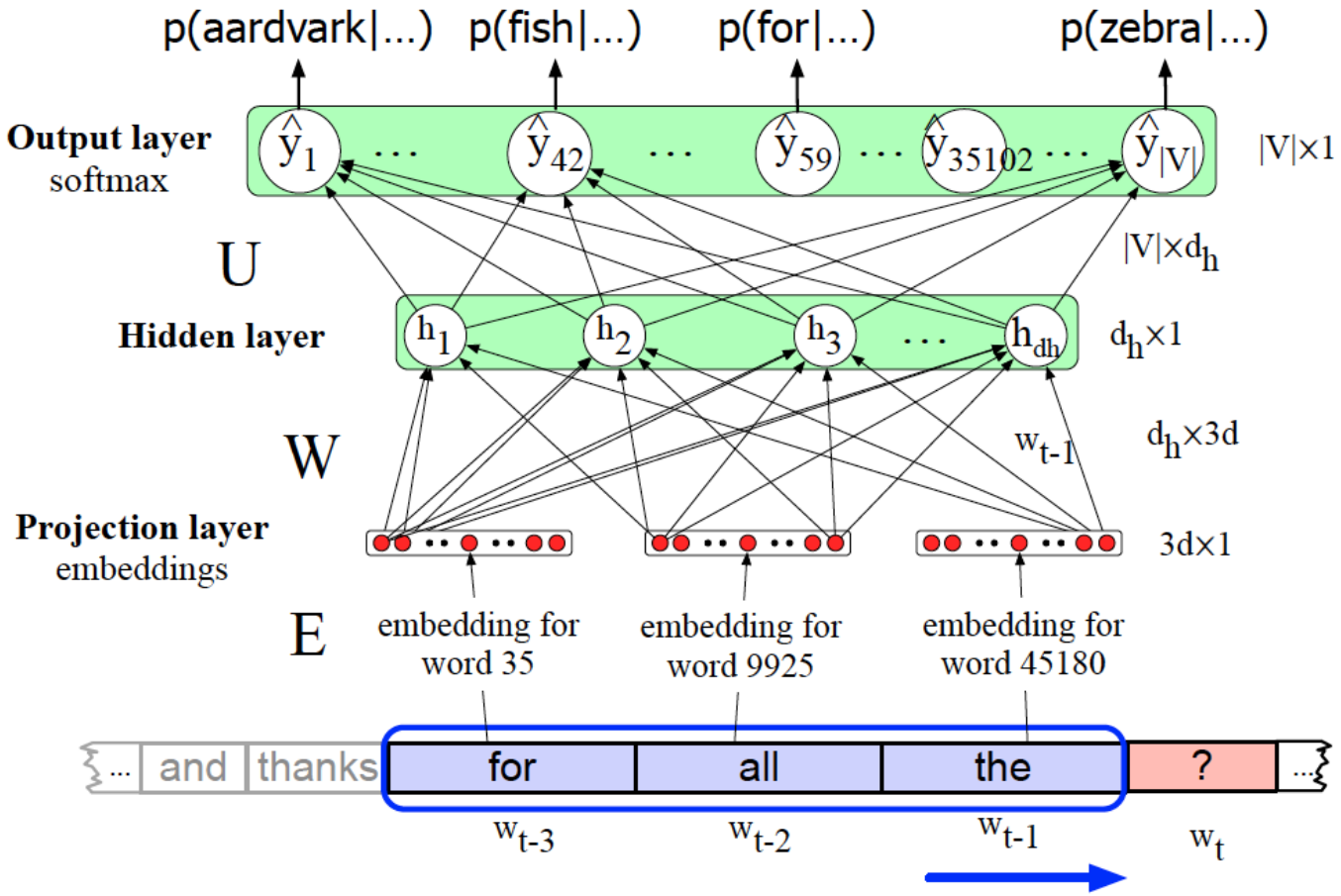
- **Problem**:

  Now we're dealing with sequences of arbitrary length

- **Solution**: Sliding windows (of fixed length)

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

Neural Language Model



Why Neural LMs work better than N-gram LMs

- **Training data:**
  - We've seen: I have to make sure that the cat gets fed
  - Never seen: dog gets fed

- **Test data:**
  - I forgot to make sure that the dog gets ___
  - N-gram LM can't predict "fed"!
  - Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict “fed” after dog